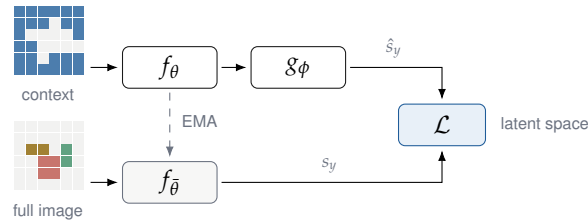


From Scratch

Joint-Embedding Predictive Architectures,
derived and then built



A self-study curriculum built from the primary literature

Prepared for Kelvin · September 2, 2026

This book is a study companion, not a substitute for the papers. Every empirical claim carries a footnote pointing at the page or table it came from, so you can disagree with me. Where the published paper and the released code disagree — and they do, in four places that matter — both are stated and the difference is flagged.

Runnable code lives in [JEPA/code/](#). Four interactive pages live in [JEPA/interactive/](#). Chapter cross-references to them are marked **OPEN**.

How to read this

You said you know roughly how neural networks and LLMs work. That is exactly the right starting point, and it means three things about how this is written.

Attention gets a refresher, not a course. Chapter 3 re-derives self-attention in one page, in the specific form I-JEPA needs, and then moves on. If it already feels obvious, skip it.

The hard part is not the architecture. I-JEPA’s diagram fits on a napkin. What takes work is understanding *why it does not degenerate*, and that is a question about optimisation dynamics, not about layers. Chapters 4–6 are the mathematical centre of the book and the longest stretch of real derivation. If you only read three chapters, read those.

You are meant to argue with it. Four kinds of box interrupt the text:

STOP AND THINK	A question you should be able to answer from what you have just read. Answer it before reading on. Answers begin on page 32.
TRAP	A mistake that is easy to make and hard to detect. Several of these cost researchers weeks.
EXERCISE	Something to run, break, or derive. Most take under 20 minutes.
Key idea	The one sentence to carry forward.

A compute plan for your hardware

You have an M4 Pro and an RTX 3060. Both are enough, because the whole book targets a ~3 M-parameter ViT on CIFAR-10 rather than a ViT-Huge on ImageNet. Concretely:

what	command	RTX 3060	M4 Pro (MPS)
shape check	<code>pretrain.py --smoke</code>	20 s	30 s
collapse lab (Ch. 6)	<code>collapse_lab.py</code>	~8 min	~18 min
full pretrain (Ch. 10)	<code>pretrain.py --epochs 60</code>	~25 min	~55 min
evaluation (Ch. 11)	<code>probe.py</code>	~2 min	~4 min

Use the 3060 for the collapse lab, where you want several runs back to back. Either machine is fine for everything else. Nothing in this book needs a cloud GPU, and nothing in it costs money.

Order of operations

Read Chapters 1–2 for the argument, 3 as needed, then *stop reading and run the collapse lab* before you read Chapter 5. The theory in Chapter 5 is much easier to absorb once you have watched a model collapse on your own screen and seen its loss go to zero while doing it.

Contents

Part I. Why predict in latent space	
1 Three ways to learn without labels	2
2 Why not predict pixels	4
3 The three things you need from transformers	6
Part II. The obstacle: collapse	
4 Collapse is a global minimum	8
5 Why stop-gradient and EMA actually work	11
6 Measuring collapse before it costs you a week	15
Part III. Building I-JEPA	
7 The architecture, precisely	17
8 Masking is the method	19
9 Writing the encoder and the predictor	21
10 The training step	23
11 Evaluating without fooling yourself	25
Part IV. Beyond images	
12 V-JEPA: the same recipe on video	27
13 V-JEPA 2, and the predictor as a world model	29
Part V. Reference	
14 What is genuinely unsettled	31
15 Answers, and where to go next	32

PART I

Why predict in latent space

Three ways to learn without labels

Start with the problem, not the architecture. You have a hundred million images and no labels. You want an encoder f such that $f(\text{image})$ is a vector you can hang a linear classifier on, or plan with, or fine-tune from. What objective do you train it on?

Everything in self-supervised vision is an answer to that question, and I-JEPA's contribution is easiest to see if you first lay out the answers as a taxonomy. The paper does this in the language of *energy-based models*, so we will too — it costs one definition and buys a lot of clarity.

1.1 The energy frame

An energy-based model is a scalar-valued function $F_w(x, y)$ that scores how compatible two things are: low energy means compatible, high energy means not.¹ It is not a probability; there is no normalising constant to compute. That is the point: you get to specify a shape for the landscape without ever integrating over it.

Training such a model has an easy half and a hard half. The easy half is pushing the energy *down* on real pairs (x, y) drawn from data. The hard half is making sure the energy is *higher* everywhere else. Miss that second half and you get what LeCun calls a collapse: “given an x the energy landscape could become ‘flat’, giving essentially the same energy to all values of y .”² Hold onto that word. Chapters 4–6 are about nothing else.

With that vocabulary, the three families:

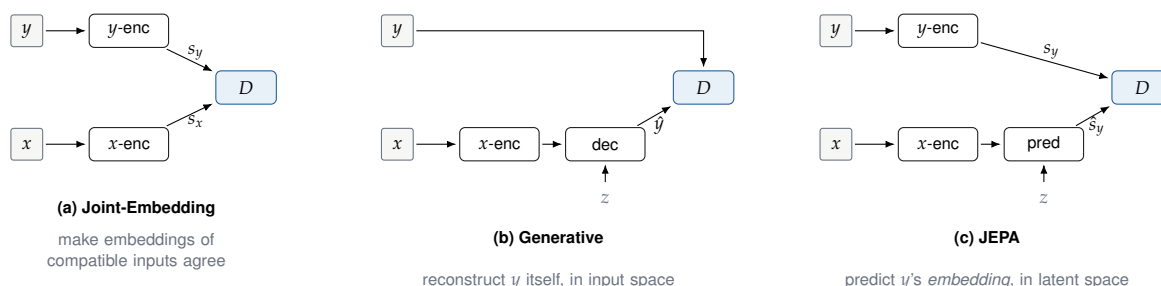


Figure 1.1. The three architecture families, redrawn from Figure 2 of the I-JEPA paper. All three push D down on compatible pairs. They differ in *where* the comparison happens and therefore in *how* they can degenerate.

(a) Joint-embedding: make two views agree

Take an image, apply two random augmentations, demand that the two embeddings match. SimCLR, MoCo, BYOL, DINO, SwAV all live here. It works well and it has two costs. The first is that you must hand-design the augmentations, and by doing so you hand-design what the representation is allowed to *forget*. Colour jitter says colour does not matter; if your downstream task is bird species identification, colour matters enormously. The second is that image augmentations do not generalise — there is no obvious “random crop” for audio spectrograms or accelerometer traces.

The failure mode is the one named above: nothing in “make the two embeddings agree” prevents f from being constant.

(b) Generative: reconstruct the input

Mask out patches, reconstruct their pixels. This is MAE, and by extension it is what BERT and GPT do to tokens. Collapse is not a worry here — the paper’s condition is precise: representation collapse is not a concern “as long as the informational capacity of z is low compared to the signal y .” If the decoder is only told *which* patch to paint and not what is in it, it cannot cheat.

The cost is different and subtler, and Chapter 2 is devoted to it.

¹Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, F. Huang, *A Tutorial on Energy-Based Learning*, 2006. The I-JEPA paper adopts this framing in its Section 2. <https://arxiv.org/abs/2301.08243>

²Y. LeCun, *A Path Towards Autonomous Machine Intelligence*, v0.9.2, 2022, §4.3, p. 20. <https://openreview.net/pdf?id=BZ5a1r-kVsF>

(c) JEPA: predict the embedding

Same masking as (b). Same predictor conditioned on positions. One change:

The whole idea, in one sentence

A JEPA is a generative architecture with the loss moved out of pixel space and into representation space. Quoting the paper directly: JEPAs are “conceptually similar to Generative Architectures; however, a key difference is that the loss function is applied in embedding space, not input space.”

That one change buys the advantages of (b) — no hand-crafted augmentations, modality-agnostic — while removing the requirement to reproduce detail. And it buys back the problem of (a), because now both branches are learned and both can go constant. The paper is candid about this: “as with Joint-Embedding Architectures, representation collapse is also a concern with JEPAs; we leverage an asymmetric architecture between the x - and y -encoders to avoid representation collapse.”

STOP AND THINK 1.1 Architecture (b) is safe from collapse when z carries little information. Architecture (c) uses the *same* z — position tokens — and is not safe. What changed? Name the specific thing that (c) can do to reach zero loss that (b) cannot.

1.2 Where this came from, and why anyone cares

The name and the argument are LeCun’s, from a 2022 position paper that proposed JEPA as a component of an autonomous-agent architecture rather than as an image model.³ I-JEPA (2023) was the first paper to make it work at scale on images; V-JEPA (2024) moved it to video; V-JEPA 2 (2025) added actions and used the result to plan real robot motion. Chapters 12–13 cover that arc.

The practical reason to care is that the predictor of a JEPA is, structurally, a world model: give me a state embedding and something to condition on, and I will tell you the next state embedding. Swap “position of a masked patch” for “action taken by a robot arm” and you have changed nothing architecturally and everything functionally.

The headline efficiency claim is worth recording early because it is what makes the method interesting rather than merely elegant: a ViT-Huge/14 pretrained on ImageNet in under 72 hours on 16 A100s, which the authors put at “over $10\times$ more efficient than a ViT-H/14 pretrained with MAE.”⁴

EXERCISE 1.1 Before reading on, write down — in one sentence each — what you predict the failure mode of each family is, and one downstream task where each family’s inductive bias would *hurt*. Keep the note. You will check it against Chapter 2’s Table 2.1 and Chapter 11’s evaluation numbers.

³LeCun 2022, op. cit., §4.4, p. 24, where the JEPA energy is written $E_w(x, y, z) = D(s_y, \text{Pred}(s_x, z))$.

⁴I-JEPA, abstract and §1. 16 GPUs $\times$ 72 h = 1152, consistent with §1’s “less than 1200 GPU hours”.

Why not predict pixels

The intuitive argument against pixel reconstruction is that most pixels are boring. It is a good intuition and it is not an argument. This chapter turns it into one, and then checks the argument against the paper’s own controlled experiment.

2.1 The conditional-variance argument

Fix an image y and a context x (say, the unmasked part). Let T be any target function of y — pixels, or an embedding, or anything else — and let $\hat{T}(x)$ be our prediction. With squared loss, the best possible predictor is the conditional mean, and its error decomposes as

$$\min_{\hat{T}} \mathbb{E} \left[\|\hat{T}(x) - T(y)\|^2 \right] = \underbrace{\mathbb{E}_x [\text{tr Var}(T(y) \mid x)]}_{\text{irreducible}}. \quad (2.1)$$

The right-hand side is the part of $T(y)$ that x simply does not determine. No model, no matter how large, can drive it down. What a model *can* do is spend capacity trying.

Now compare two choices of T on a masked patch of grass.

- T = raw pixels. The exact arrangement of blades of grass is not determined by the surrounding image. $\text{tr Var}(T \mid x)$ is enormous, and it is enormous for reasons that carry no semantic content. Every gradient step is partly a step towards fitting noise.
- T = a learned embedding. Now T is under no obligation to retain blade positions. If the encoder maps “grass texture” to a single code, the conditional variance of *that* code given the context is small, and the loss becomes a pure test of whether the model knows there is grass there.

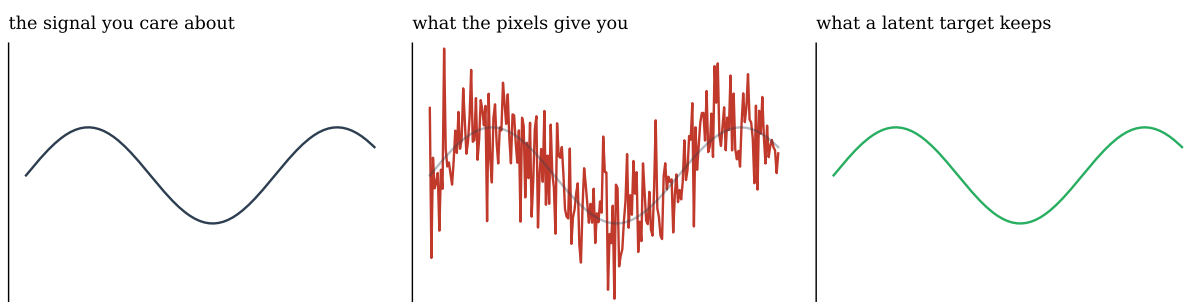


Figure 2.1. The same idea as a signal. A pixel target hands the model signal *plus* unpredictable detail and scores it on both. A learned latent target is free to discard the detail, so the loss measures only what was predictable in principle.

TRAP

This is not the claim that pixel prediction is bad. It is the claim that pixel prediction spends capacity on a term that Eq. (2.1) says is unwinnable. MAE representations are genuinely good — they are simply good in a different way, and need more adaptation to become linearly separable. On the paper’s own local-prediction benchmarks MAE *beats* I-JEPA: Clevr/Count 90.5 versus 86.7.^a

^aI-JEPA Table 4. MAE ViT-H/14 vs I-JEPA ViT-H/14, linear probe. The paper’s prose claims superiority only over view-invariance methods here, and is careful about it.

2.2 The catch, stated honestly

Equation (2.1) has a hole in it. If T is *learned*, then T can make the irreducible term zero by being constant. The argument for latent targets and the problem of collapse are the same fact seen from two sides:

Key idea

Predicting in latent space works because the target is allowed to throw information away. It fails — collapses — for exactly the same reason. The entire engineering problem of JEPA is to permit the first without permitting the second.

STOP AND THINK 2.1 A generative architecture has a decoder and cannot collapse; a JEPA has no decoder and can. Yet the JEPA’s *predictor* and the generative model’s *decoder* receive the same conditioning information. Explain the asymmetry in one sentence, using the phrase “informational capacity”.

2.3 What the controlled experiment says

The paper runs the ablation that this chapter’s argument predicts: same architecture, same masking, same everything, with the loss computed against the target encoder’s output versus against raw pixels.

prediction target	arch.	epochs	IN-1% top-1
target-encoder output	ViT-L/16	500	66.9
pixels	ViT-L/16	800	40.7

Table 2.1. Predicting representations versus predicting pixels, linear probe on 1% of ImageNet labels.¹

A 26-point gap, with the losing configuration given 60% more training. That is about as clean as ablations in this literature get.

The complementary evidence is qualitative. After pretraining, the authors froze the encoder and predictor and trained a diffusion decoder (RCDM) to map the predictor’s pooled output back to pixels, then sampled it many times. Features that are stable across samples are in the representation; features that vary are not. What they report is that the predictor produces “high-level object parts with a correct pose (e.g., the back of the bird and top of a car)” while discarding “the precise low-level details as well as background information.”² That is precisely the behaviour Eq. (2.1) recommends: keep what the context determines, drop what it does not.

EXERCISE 2.1 Derive Eq. (2.1). Start from $\mathbb{E}\|\hat{T}(x) - T(y)\|^2$, insert $\pm \mathbb{E}[T(y) \mid x]$, and show the cross term vanishes. Then answer: what changes if the loss is ℓ_1 rather than ℓ_2 ? (V-JEPA switched to ℓ_1 , calling it “more stable” — so this is not an idle question.)³

²I-JEPA §8 and Figure 6 caption; the RCDM procedure is detailed in Appendix E, trained 300k iterations.

³V-JEPA, *Revisiting Feature Prediction for Learning Visual Representations from Video*, §3.1: “we modify it to use an ℓ_1 regression, which we found to be more stable.” <https://arxiv.org/abs/2404.08471>

The three things you need from transformers

You know transformers from language models. This chapter is a targeted refresher: only the three facts I-JEPA leans on, and one thing that is genuinely different from the LLM setting. Skip it if it reads as obvious.

3.1 Patches are tokens

A ViT cuts a $H \times W$ image into a grid of non-overlapping $P \times P$ patches and treats each as a token.¹ For 224×224 input and $P = 16$ that is a 14×14 grid, $N = 196$ tokens. For the CIFAR-sized model you will build, 32×32 with $P = 4$ gives an 8×8 grid, $N = 64$.

The patch embedding is a single strided convolution — kernel P , stride P — which is exactly “flatten each patch and apply one shared linear map”:

```
self.patch_embed = nn.Conv2d(3, dim, patch_size, stride=patch_size)
x = self.patch_embed(x).flatten(2).transpose(1, 2)  # [B, 3, H, W] → [B, N, dim]
```

Flattening is row-major, so token k sits at grid position $(k \div G, k \bmod G)$ where G is the grid side. That convention appears in four separate places in the code and every one of them has to agree; getting it wrong produces a model that trains happily and means nothing.

3.2 Attention is a soft lookup, and it is set-valued

For each token, project to a query, a key and a value; score every query against every key; softmax; take the weighted average of values:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right)V, \quad Q = XW_Q, K = XW_K, V = XW_V. \quad (3.1)$$

Two consequences matter here.

It is permutation-equivariant. Attention has no idea where tokens are. Position enters only through what you add to the token embeddings. In an LLM you usually use RoPE or learned positions; I-JEPA uses *fixed* 2-D sinusoidal embeddings, marked `requires_grad=False`. The paper never says which kind — only that mask tokens are “a shared learnable vector with an added positional embedding” — so this is a detail you can only get from the released code.

It costs nothing to drop tokens. Because attention operates on a set, you can simply *not include* the masked patches. This is why an I-JEPA context encoder is cheap: the paper notes the context encoder “only processes the visible context patches.” With a 25% context you pay roughly a sixteenth of the attention cost, since attention is quadratic in sequence length.

3.3 No causal mask, and no [CLS]

Here is the real difference from your LLM mental model. An LLM masks attention so token t cannot see $t + 1$. I-JEPA does *none* of that. Both the encoder and the predictor use full bidirectional attention. Inside the predictor, the mask tokens attend to the context *and to each other*.

There is also no [CLS] token. Global representations are obtained by average-pooling the patch tokens: “I-JEPA is pretrained without a [cls] token. We use the target-encoder for evaluation and average pool its output to produce a global image representation.”²

TRAP

Sinusoidal position tables interact badly with changing resolution. The released I-JEPA code contains an `interpolate_pos_encoding` function that computes its lengths as if a [CLS] token existed; because both quantities are off by the same one, the function is a no-op at native resolution and silently wrong otherwise. If you ever fine-tune at a different input size, write your own.

STOP AND THINK 3.1 The predictor gives every mask token the *same* learnable vector, plus a positional embedding. If you deleted the positional embedding and kept the shared vector, what would the predictor output, and why

¹A. Dosovitskiy et al., *An Image is Worth 16x16 Words*, ICLR 2021. <https://arxiv.org/abs/2010.11929>

²I-JEPA Appendix A.1, *Architectures*. This sentence also settles the question of *which* encoder to evaluate — see Chapter 11.

is that answer forced by the previous section?

EXERCISE 3.1 Convince yourself the encoder really is cheaper. For $G = 14$ ($N = 196$) and a context ratio of 0.25, compute the ratio of attention FLOPs ($\propto n^2 d$) between encoding the full grid and encoding only the context. Then check that against the paper’s claim that I-JEPA is “about 7% slower time per iteration” than pixel prediction — what is the extra 7% paying for?

PART II

The obstacle: collapse

Collapse is a global minimum

Most training pathologies are bad local minima, or bad conditioning, or a bad learning rate. Collapse is none of those. It is the loss function working exactly as specified. This chapter proves that, then surveys the four ways the field has found around it.

4.1 The proposition

Let $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$ be an encoder and let p_{pos} be a distribution over compatible pairs — two augmentations of one image, or a context and a target block. Consider the pure invariance objective

$$\mathcal{L}_{\text{inv}}(\theta) = \mathbb{E}_{(x,y) \sim p_{\text{pos}}} [\|f_\theta(x) - f_\theta(y)\|_2^2]. \quad (4.1)$$

Proposition. $\mathcal{L}_{\text{inv}} \geq 0$, being an expectation of a squared norm. If the hypothesis class contains any constant map — if there exists θ_0 with $f_{\theta_0} \equiv c$ — then $\mathcal{L}_{\text{inv}}(\theta_0) = 0$, which attains the lower bound. Hence θ_0 is a *global* minimiser, and the whole family $\{f \equiv c : c \in \mathbb{R}^d\}$ is a d -parameter continuum of global optima.

The proof is the two sentences above. What matters is the corollary.

Corollary 1: this is not an optimisation problem

Collapse cannot be fixed by a better optimiser, a longer schedule, more compute, or a warmup. The collapsed solution is a global minimum of the objective you wrote down. Every non-collapsing method must do one of exactly two things:

1. **Change the objective**, so that constants are no longer minimisers. Contrastive, redundancy-reduction and clustering methods all do this.
2. **Change the dynamics**, leaving the objective’s minimisers untouched but making the collapsed one unreachable from a typical initialisation. BYOL, SimSiam and I-JEPA do this.

I-JEPA is firmly in camp 2, which is why Chapter 5 has to be about differential equations rather than about loss design.

Corollary 2: normalising does not help

You might hope that ℓ_2 -normalising the outputs rules it out. It does not. With $\mathcal{D}(p, z) = -\frac{p}{\|p\|} \cdot \frac{z}{\|z\|}$ the minimum is exactly -1 , attained by any constant *direction*. SimSiam reports it happening in one line: “Without stop-gradient, the optimizer quickly finds a degenerated solution and reaches the minimum possible loss of -1 .”¹

4.2 The weaker, sneakier version

Complete collapse is loud. The version that will actually cost you a week is *dimensional* collapse: the embeddings do not shrink to a point, they shrink to a low-dimensional subspace of $\mathbb{R}^d$. Formally, with

$$C = \frac{1}{N} \sum_{i=1}^N (z_i - \bar{z})(z_i - \bar{z})^\top \in \mathbb{R}^{d \times d}, \quad C = USV^\top, \quad S = \text{diag}(\sigma^1, \dots, \sigma^d), \quad (4.2)$$

complete collapse means $\text{rank}(C) = 0$ and dimensional collapse means $0 < \text{rank}(C) < d$ — “the embedding vectors only span a lower-dimensional subspace.”

The striking empirical fact is that this happens even to methods that *provably* cannot completely collapse. For a pretrained SimCLR with $d = 128$, evaluated on ImageNet validation, “about 30 singular values drop to zero” — roughly a quarter of the embedding space is dead.²

That paper identifies two independent causes, and the second is the surprising one. Cause 1 is strong augmentation: if the augmentation covariance dominates the data covariance, a certain matrix $X = \hat{\Sigma}_0 - \hat{\Sigma}_1$ acquires negative eigenvalues and the corresponding singular values of the weights decay to zero. Cause

¹X. Chen, K. He, *Exploring Simple Siamese Representation Learning*, CVPR 2021, §4.1. <https://arxiv.org/abs/2011.10566>

²L. Jing, P. Vincent, Y. LeCun, Y. Tian, *Understanding Dimensional Collapse in Contrastive Self-supervised Learning*, ICLR 2022, Fig. 2; the singular-value dynamics below are their Theorem 3, Eqs. (10)–(11). The $\text{rank}(C)$ phrasing is mine — they write only “the embedding vectors only span a lower-dimensional subspace”. <https://arxiv.org/abs/2110.09348>

2 needs no strong augmentation at all — it is a pure effect of *depth*. For a two-layer linear net with aligned weights, the singular values obey

$$\dot{\sigma}_1^k = \sigma_1^k (\sigma_2^k)^2 (v_1^{k\top} X v_1^k), \quad \dot{\sigma}_2^k = \sigma_2^k (\sigma_1^k)^2 (v_1^{k\top} X v_1^k), \quad (4.3)$$

which is *cubic* in σ . Small singular values therefore grow much more slowly than large ones, so the spectrum spreads out on its own. A single linear layer with $X \succ 0$ shows no dimensional collapse; adding layers produces more collapsed directions.

STOP AND THINK 4.1 Corollary 1 says collapse is a global minimum of $\mathcal{L}_{\text{inv}}$. Dimensional collapse is *not* a global minimum of the SimCLR objective — the negatives penalise it. So what causes it, in one word? And which of Corollary 1’s two remedies could possibly address it?

4.3 The four escapes

Here is the field, compressed. Each row changes the objective (rows 1, 3, 4) or the dynamics (row 2).

1. Contrastive: add negatives

SimCLR’s NT-Xent, for a positive pair (i, j) in a batch of N images ($2N$ views):

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}, \quad \text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}. \quad (4.4)$$

A constant map now scores $\ell = \log(2N - 1) > 0$, so it is removed from the minimiser set outright.³ The cost is the number of negatives. LeCun’s objection is dimensional: “in the worst case, the number of contrastive samples may grow exponentially with the dimension of the representation. This is the main reason why I will argue against contrastive methods.”⁴

2. Asymmetry: stop-gradient, plus optionally EMA

BYOL keeps two networks. The online parameters θ take gradient steps; the target parameters ξ do not, moving only by exponential moving average:

$$\xi \leftarrow \tau \xi + (1 - \tau) \theta, \quad \mathcal{L}_{\theta, \xi} = \|\overline{q_\theta(z_\theta)} - \overline{z'_\xi}\|_2^2, \quad (4.5)$$

minimised *with respect to θ only*.⁵ The constant solution is still a minimum of $\mathcal{L}$; the system simply never goes there. BYOL’s own explanation is a confession of ignorance, and an honest one: they “hypothesize that there is no loss $\mathcal{L}_{\theta, \xi}$ such that BYOL’s dynamics is a gradient descent on $\mathcal{L}$ jointly over θ, ξ ”, hence “no a priori reason why BYOL’s parameters would converge to a minimum.” Chapter 5 supplies the missing analysis.

This is I-JEPA’s row. Note the paper’s own hedge, which is worth reading carefully: “The use of an exponential moving average target-encoder has proven essential for training JEAs with Vision Transformers, we find the same to be true for I-JEPA.” That is an appeal to prior empirical work, not a mechanism — and I-JEPA contains no EMA ablation and no stop-gradient ablation.

3. Redundancy reduction: regularise the statistics

VICReg attacks it head-on with three terms on a batch of embeddings $Z \in \mathbb{R}^{n \times d}$:

$$v(Z) = \frac{1}{d} \sum_{j=1}^d \max(0, \gamma - \sqrt{\text{Var}(z^j) + \epsilon}) \quad \text{variance (anti-collapse)} \quad (4.6)$$

$$c(Z) = \frac{1}{d} \sum_{i \neq j} [C(Z)]_{i,j}^2 \quad \text{covariance (decorrelate)} \quad (4.7)$$

$$s(Z, Z') = \frac{1}{n} \sum_i \|z_i - z'_i\|_2^2 \quad \text{invariance} \quad (4.8)$$

with $\ell = \lambda s + \mu[v(Z) + v(Z')] + \nu[c(Z) + c(Z')]$, and defaults $\gamma = 1, \epsilon = 10^{-4}, \lambda = \mu = 25, \nu = 1$.⁶ Two details repay attention. First, the hinge is on the standard *deviation*, not the variance, and the paper says why: with $S(x) = \text{Var}(x)$, “the gradient of S with respect to x becomes close to 0 when x is close to $\bar{x}$... and

³T. Chen, S. Kornblith, M. Norouzi, G. Hinton, *A Simple Framework for Contrastive Learning of Visual Representations*, ICML 2020, Eq. 1. <https://arxiv.org/abs/2002.05709>

⁴LeCun 2022, §4.3, p. 23. He also notes, and this is usually dropped in summary, that “contrastive and regularized methods are not incompatible with each other, and can be used simultaneously on the same model.”

⁵J.-B. Grill et al., *Bootstrap Your Own Latent*, NeurIPS 2020, Eqs. 1–3. <https://arxiv.org/abs/2006.07733>

⁶A. Bardes, J. Ponce, Y. LeCun, *VICReg*, ICLR 2022, Eqs. 1–7. <https://arxiv.org/abs/2105.04906>

the embeddings collapse.” Compare $\partial_{x_i} \text{Var}(x) = \frac{2}{n}(x_i - \bar{x}) \rightarrow 0$ against $\partial_{x_i} \sqrt{\text{Var} + \epsilon} = \frac{(x_i - \bar{x})/n}{\sqrt{\text{Var} + \epsilon}}$, which stays bounded away from zero. Second, their ablation shows invariance *plus* covariance still collapses — decorrelation alone does not stop shrinkage, because the covariance of a constant is zero and therefore trivially diagonal.

Barlow Twins is the same idea in a tighter package — an invariance term and a redundancy-reduction term, driving the cross-correlation matrix towards the identity: $\mathcal{L} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2$ with $\lambda = 5 \times 10^{-3}$, correlations taken *along the batch dimension* rather than along features. Their ablation is as sharp as VICReg’s: delete the off-diagonal term and top-1 falls from 71.4 to 0.1.⁷

This is the family LeCun advocates, and he is explicit that it is *not* the BYOL family: “a number of new non-contrastive methods have appeared, such as BYOL. But the class of non-contrastive methods advocated in the present proposal prevent collapse by maximizing the information content of the embeddings.”

4. Clustering: force an equipartition

DINO’s teacher output is centred then sharpened, $\text{softmax}((g_{\theta_t}(x) - c)/\tau_t)$ with $c \leftarrow mc + (1 - m)\frac{1}{B} \sum_i g_{\theta_t}(x_i)$, and the two operations oppose each other on purpose: “centering prevents one dimension to dominate but encourages collapse to the uniform distribution, while the sharpening has the opposite effect. Applying both operations balances their effects.”⁸

TRAP

The BatchNorm story is settled and is still widely misremembered. A well-read 2020 blog post argued BN was doing implicit contrastive learning inside BYOL by leaking batch statistics. BYOL’s authors then replaced BN with a batch-independent scheme (group norm + weight standardisation) and reached 73.9% versus vanilla BYOL’s 74.3% on ImageNet, concluding this “disproves the hypothesis that the use of batch statistics is a crucial ingredient.” Their diagnosis is that BN’s real job is conditioning at initialisation, not contrast.^a Existence proof: I-JEPA has no BatchNorm anywhere and does not collapse.

^aP. Richemond et al., *BYOL works even without batch statistics*, 2020, abstract and Table 2. <https://arxiv.org/abs/2010.10241> SimSiam independently agrees: removing all BN gives 34.6%, and “this variant does not cause collapse, although the accuracy is low”. DINO with a ViT is BatchNorm-free entirely.

EXERCISE 4.1 Prove the analogue of the Proposition for NT-Xent (Eq. 4.4): show that the constant map gives exactly $\ell = \log(2N - 1)$, and then show this is *not* the minimum by exhibiting any configuration with a lower value. What happens to your bound as $\tau \rightarrow 0$?

⁷J. Zbontar, L. Jing, I. Misra, Y. LeCun, S. Deny, *Barlow Twins*, ICML 2021, Eqs. 1–2. <https://arxiv.org/abs/2103.03230>

⁸M. Caron et al., *Emerging Properties in Self-Supervised Vision Transformers (DINO)*, ICCV 2021, §3.1, Eqs. 2–4. <https://arxiv.org/abs/2104.14294> Thresholds matter, and they are in Appendix D rather than §3.1: $\tau_t < 0.06$ is required, and $m = 0.999$ collapses.

Why stop-gradient and EMA actually work

This is the chapter the \$500 course spends a lecture on and most tutorials skip. The claim “EMA and stop-gradient prevent collapse” is usually asserted. It can be *derived*, in a linear model, and the derivation predicts things you can check — including three predictions that were later confirmed experimentally. Everything below follows Tian, Chen and Ganguli.¹ I have integrated their equations numerically so that you can see the mechanism rather than take it on faith.

5.1 The setup

Strip everything down to linear maps: an online network W , a predictor W_p , a target network W_a , no biases, no nonlinearities. Views x_1, x_2 of the same input; $f_1 = Wx_1$; $f_{2a} = W_ax_2$. The objective, with the stop-gradient written explicitly, is

$$J(W, W_p) = \frac{1}{2} \mathbb{E} [\|W_p f_1 - \text{sg}(f_{2a})\|_2^2]. \quad (5.1)$$

Two data matrices appear: $X = \mathbb{E}[\tilde{x}\tilde{x}^\top]$, the covariance of the *mean* augmented view, and $X' = \mathbb{E}_x[\text{Var}_{x'|x}(x')]$, the mean augmentation covariance. Write σ^2 for the scale of X' when the data is isotropic. Gradient flow gives (their Lemma 1)

$$\dot{W}_p = \alpha_p (-W_p W(X + X') + W_a X) W_p^\top - \eta W_p, \quad (5.2)$$

$$\dot{W} = W_p^\top (-W_p W(X + X') + W_a X) - \eta W, \quad (5.3)$$

$$\dot{W}_a = \beta(-W_a + W), \quad (5.4)$$

where α_p is the predictor’s relative learning rate, η is weight decay, and β is the EMA rate ($\alpha\beta = 1 - \gamma_a$, with $\gamma_a = 0.996$ in BYOL and in I-JEPA). Setting $W_a = W$ and deleting the third equation gives SimSiam — the stop-gradient is still there.

5.2 Result 1: without stop-gradient, collapse is provable

Their Theorem 2 removes the stop-gradient in the SimSiam case. The dynamics become

$$\frac{d}{dt} \text{vec}(W) = -H(t) \text{vec}(W), \quad H = X' \otimes (W_p^\top W_p + I) + X \otimes \tilde{W}_p^\top \tilde{W}_p + \eta I, \quad (5.5)$$

with $\tilde{W}_p = W_p - I$. Every term is positive semi-definite. If $\inf_t \lambda_{\min}(H) \geq \lambda_0 > 0$ then $W(t) \rightarrow 0$ — exponentially.

The scalar case makes this vivid. Take $n_1 = n_2 = 1$, $W = w$, $W_p = p$, $X = 1$, $X' = \sigma^2$, $\alpha_p = 1$. Then $H = \sigma^2(p^2 + 1) + (p - 1)^2 + \eta \geq \sigma^2 > 0$ for every p , so

$$\dot{w} = -Hw \implies |w(t)| \leq |w(0)|e^{-\sigma^2 t} \rightarrow 0. \quad (5.6)$$

Result 1

Without a stop-gradient, collapse is not a risk — it is the unique attractor, and the rate is set by the augmentation variance σ^2 . This is the one piece of the puzzle that is a theorem rather than a heuristic.

5.3 Result 2: with stop-gradient, collapse becomes a saddle

Now keep the stop-gradient. Assume the target is proportional to the online network, $W_a = \tau W$ (their assumption A1 — the EMA in closed form), isotropic data (A2), and symmetric W_p (A3). Define $F := W X W^\top$, the correlation matrix of the online network’s outputs, which is exactly the predictor’s *input* statistic. In the eigenbasis of F , with p_j the j -th eigenvalue of W_p and s_j that of F , the dynamics decouple

¹Y. Tian, X. Chen, S. Ganguli, *Understanding self-supervised learning dynamics without contrastive pairs*, ICML 2021. <https://arxiv.org/abs/2102.06810> Equations here are numbered as this book’s; the correspondence is 5.1=their 1, 5.2–5.4=2–4, 5.5=6, 5.7/5.8=11/12, 5.9=14, 5.10=16, 5.12=8 and 10, 5.13=17. All numerical results in Figures 5.1 and 5.2 are my own integration of their equations; the script is `code/./exp/tian_dynamics.py`.

per mode:

$$\dot{p}_j = \alpha_p s_j [\tau - (1 + \sigma^2) p_j] - \eta p_j, \quad (5.7)$$

$$\dot{s}_j = 2p_j s_j [\tau - (1 + \sigma^2) p_j] - 2\eta s_j. \quad (5.8)$$

Two branches of fixed points fall straight out: the *collapsed* branch $s_j = 0$ with p_j arbitrary, and the *useful* branch $p_j^* = \tau / (1 + \sigma^2)$ with s_j arbitrary.

At $\eta = 0$ the flow conserves a quantity,

$$s_j(t) = \alpha_p^{-1} p_j^2(t) + e^{-2\eta t} c_j, \quad c_j = \alpha_p^{-1} p_j^2(0) - s_j(0), \quad (5.9)$$

so trajectories are confined to a parabola $s = p^2 / \alpha_p + c$. Starting *above* the parabola through the origin, i.e. $s_j(0) > p_j^2(0) / \alpha_p$, is sufficient to escape.

STOP AND THINK 5.1 Equation (5.7) has s_j multiplying everything. What does that say about the collapsed branch $s_j = 0$ — how fast does p_j move there? Now explain, in one sentence, why the collapsed branch being a *saddle* rather than a repeller is enough for training to work in practice but not enough to prove it always will.

5.4 Result 3: weight decay creates a genuine trap

Switch weight decay on. The $e^{-2\eta t} c_j$ term in Eq. (5.9) dies, so the flow lands on the invariant parabola $s_j = \alpha_p^{-1} p_j^2$. Substituting into Eq. (5.7) gives a one-dimensional flow,

$$\dot{p}_j = p_j \Delta_j, \quad \Delta_j := p_j [\tau - (1 + \sigma^2) p_j] - \eta, \quad (5.10)$$

whose fixed points are $p_{j0}^* = 0$ and

$$p_{j\pm}^* = \frac{\tau \pm \sqrt{\tau^2 - 4\eta(1 + \sigma^2)}}{2(1 + \sigma^2)}. \quad (5.11)$$

For $0 < \eta \leq \frac{\tau^2}{4(1 + \sigma^2)}$ there are three: 0 and p_{j+}^* are stable, p_{j-}^* is unstable. So the collapse basin is $p_j < p_{j-}^*$. Above the critical decay $\eta_{\text{crit}} = \tau^2 / [4(1 + \sigma^2)]$ the only fixed point left is zero and, in their words, “under such strong weight decay collapse is unavoidable.”

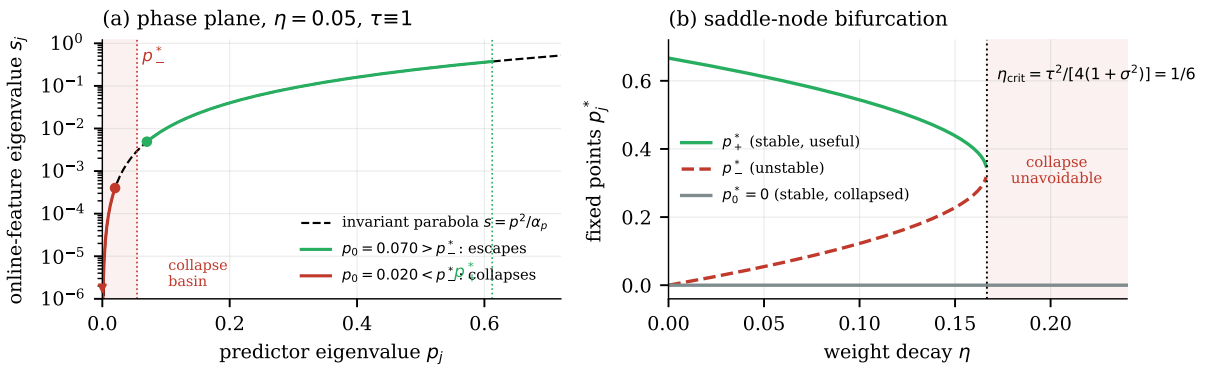


Figure 5.1. Numerical integration of Eqs. (5.7)–(5.8) at $\sigma^2 = 0.5$, $\alpha_p = 1$. **(a)** With $\eta = 0.05$ theory predicts $p_{j-}^* = 0.05445$. Integrating from exactly on the parabola: $p_0 = 0.0540$ collapses, $p_0 = 0.0548$ escapes to $p_{j+}^* = 0.61222$. The predicted and observed boundaries agree at every one of seven test points. **(b)** The saddle-node bifurcation. At $\eta > \eta_{\text{crit}} = 1/6$ the useful branch ceases to exist. That agreement is worth pausing on. This is a closed-form prediction about *which initialisations of a predictor lead to a dead representation*, and it is right to four significant figures.

5.5 Result 4: what the predictor is for

There is a second theorem, and it explains a piece of the architecture that otherwise looks arbitrary. Let $[A, B] = AB - BA$. Their Theorem 3 shows

$$\frac{d}{dt} [F, W_p] = -[F, W_p]K - K[F, W_p], \quad \|[F(t), W_p(t)]\|_F \leq e^{-2\lambda_0 t} \|[F(0), W_p(0)]\|_F, \quad (5.12)$$

whenever $\inf_t \lambda_{\min}(K) = \lambda_0 > 0$. A vanishing commutator means F and W_p become simultaneously diagonalisable: *the predictor’s eigenspace aligns itself with the covariance structure of the online network’s own outputs*. The predictor is not a generic extra layer; it is the component that learns the shape of the feature distribution, which is precisely why deleting it collapses BYOL and SimSiam.

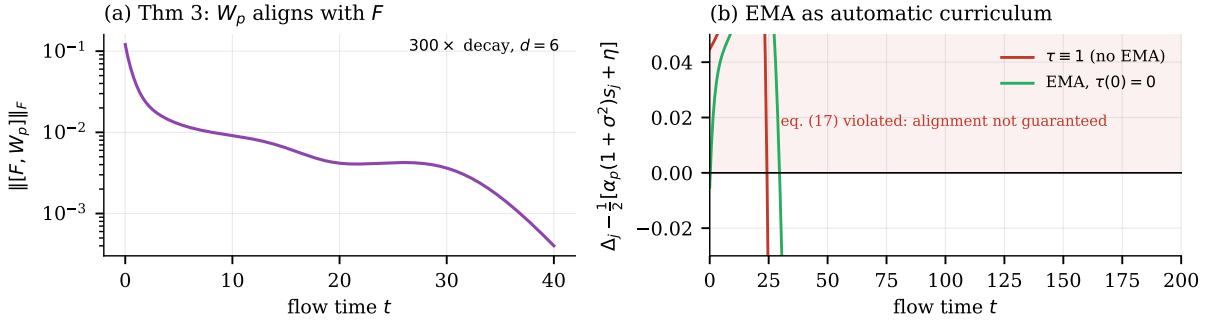


Figure 5.2. (a) My integration of their matrix system for $d = 6$, symmetric W_p , $\eta = 0.02$, $\tau = 0.8$: $\|[F, W_p]\|_F$ falls from 0.121 to 4.0×10^{-4} , and afterwards the largest off-diagonal entry of W_p in F ’s eigenbasis is 3.0×10^{-3} — simultaneously diagonal, as predicted. (b) Condition (5.13), plotted as Δ_j — bound; negative means satisfied. With $\tau \equiv 1$ it is violated for the first $\sim 24,000$ integration steps. With EMA it holds from step zero.

5.6 Result 5: EMA is an automatic curriculum

We can now say what the EMA is *for*, which is the part almost nobody states precisely. Two things must hold at once early in training: $\Delta_j > 0$, so p_j grows off zero; and, for the alignment of Result 4 to stay stable,

$$\Delta_j < \frac{1}{2}[\alpha_p(1 + \sigma^2)s_j + \eta]. \quad (5.13)$$

Look at the orders. When $\tau = 1$ and p_j is small, Δ_j is *linear* in p_j while the right-hand side is $O(s_j) \sim O(p_j^2)$. The left side wins. Condition (5.13) fails exactly in the regime where you need it.

EMA fixes this by making τ start small and grow:

Result 5 — the mechanism, in their words

“EMA can serve as an automatic curriculum: it sets an initial small goal of $\frac{\tau}{1+\sigma^2}$ for p_j so Δ_j need only be small and positive to both drive p_j larger and satisfy Eqn. 17. Then EMA gradually sets a higher goal for p_j by increasing τ , so that p_j and s_j can grow, while keeping the eigenspaces of W_p and F aligned.”

My integration bears this out numerically. Starting from $p = 0.05$, $s = 0.002$, $\eta = 0$: with $\tau \equiv 1$, condition (5.13) is violated for the first 24,278 steps and holds for 87.9% of the run; with EMA at $\beta = 0.5$ it holds from step 0. Both end at the same fixed point — EMA does not change *where* you land, it changes whether you stay aligned on the way there. Evaluating Eq. (5.10) and Eq. (5.13) at $p = 0.05$, $s = 0.002$:

τ	Δ_j	bound	p_j grows?	stays aligned?
1.00	+0.04625	0.0015	yes	no
0.30	+0.01125	0.0015	yes	no
0.05	−0.00125	0.0015	not yet	yes

Table 5.1. The trade-off EMA is navigating. Large τ grows the predictor but breaks alignment; small τ preserves alignment at the cost of growth. Ramping τ from 0 towards 1 walks the boundary.

5.7 Three predictions, all confirmed

A theory earns trust by being falsifiable. This one made three testable claims and then checked them, in the same paper — weaker evidence than independent replication, but the predictions were derived before the experiments and one of them had already been observed by BYOL’s authors a year earlier without explanation:

1. **A symmetric predictor needs EMA; an asymmetric one does not.** On STL-10, 100 epochs: symmetric W_p with EMA scores 75.09 ± 0.48 , without EMA 36.62 ± 1.85 . On ImageNet, a symmetric one-layer linear predictor in SimSiam with no EMA gives 0.1% top-1 — chance — while the asymmetric version gives 68.1%.

2. **Weight decay on the predictor alone can substitute for EMA.** $\bar{\eta}_p = 0.0004$, $\bar{\eta}_s = 0$, no EMA: 71.12 ± 0.71 , versus 43.04 ± 2.32 with zero decay everywhere. This is exactly what Eq. (5.10) says — decay reshapes the basin.
3. **A fast predictor ($\alpha_p > 1$) can substitute for EMA.** Confirmed, and consistent with BYOL’s own Appendix I.1, Table 22, which reports that “the predictor learning rate needs to be higher than the projector learning rate in order to successfully remove the target network” (3.2% at equal rates, 66.9% at $10\times$).

And SimSiam’s independent finding fits neatly into this frame: the *stop-gradient* is what prevents collapse, and the *EMA* buys accuracy. Their words: “Our empirical study challenges the necessity of the momentum encoder for preventing collapsing. We discover that the stop-gradient operation is critical.” And, of the moving average specifically: “our experiments show that it is not directly related to preventing collapsing.” Removing the stop-gradient takes SimSiam from 67.7% to 0.1%.

TRAP

Chance-level accuracy does not prove collapse. SimSiam is careful about this: “a chance-level accuracy (0.1%) is not sufficient to indicate collapsing. A model with a diverging loss, which is another pattern of failure, may also exhibit a chance-level accuracy.” Their own fixed-random-predictor ablation scores 1.5% and they explicitly say “this failure is not about collapsing. The training does not converge.” Diagnose with the spectrum, not the accuracy — which is Chapter 6.

EXERCISE 5.1 Reproduce Figure 5.1(a) yourself. Integrate Eqs. (5.7)–(5.8) with $\sigma^2 = 0.5$, $\alpha_p = 1$, $\eta = 0.05$, $\tau = 1$, starting from $s_0 = p_0^2$, and bisect on p_0 to find the collapse boundary. You should recover 0.05445. Then set $\eta = 0.2$ and confirm that *every* initialisation collapses.

EXERCISE 5.2 Take Result 3 seriously, but carefully. Result 3 says the collapse basin $p_j < p_{j-}^*$ *widens* with η , so a schedule that starts small and grows keeps the basin narrow exactly when p_j is small and vulnerable. Argue that qualitatively, then explain why you cannot plug I-JEPA’s numbers into $\eta_{\text{crit}} = \tau^2 / [4(1 + \sigma^2)]$ directly: their η is a gradient-flow coefficient with $\bar{\eta} = \alpha\eta$, so an AdamW weight decay of 0.4 at $\text{lr} = 10^{-3}$ corresponds to an η hundreds of times $1/6$. Finally, check what the empirical trade-off actually is: the paper compares its ramp against a *fixed* 0.05, and finds the ramp better for linear probing (77.8 vs 76.4) and *worse* for low-shot fine-tuning (69.4 vs 70.7).²

²I-JEPA Table 13, ViT-L/16, 600 epochs. Note also the small inconsistency: ViT-L/16 at 600 epochs is 77.5 in Table 1 and 77.8 here. The unit mismatch is Tian et al.’s Fig. 5 caption: “Here $\eta = 0.04$ so that $\eta\alpha$ equals to the weight decay ($\bar{\eta} = 0.0004$) in our STL-10 experiments.”

Measuring collapse before it costs you a week

Chapters 4 and 5 argued that the loss is not a progress signal. This chapter gives you the four numbers that are, and then shows you a real run where the loss was lower and the representation was worse.

6.1 The dashboard

Embedding standard deviation — the cheap one

Take pooled embeddings z , ℓ_2 -normalise each, and report the mean per-channel standard deviation over the batch. For a constant encoder it is exactly 0. For embeddings scattered isotropically on the unit sphere, writing $z'_i = z_i / (\sum_j z_j^2)^{1/2}$ with $z_j \sim \mathcal{N}(0, 1)$, we get $z'_i \approx z_i / \sqrt{d}$ and hence $\text{std}[z'_i] \approx 1/\sqrt{d}$.¹ Report the ratio to $1/\sqrt{d}$ so it is comparable across widths. One number, two lines of code, catches complete collapse instantly.

Singular-value spectrum — the diagnostic one

Centre the embeddings, take the SVD, plot $\log \sigma_k$ sorted. Collapsed directions are a cliff at the right-hand end. This is the only diagnostic that distinguishes complete from dimensional collapse.

RankMe — the scalar summary

Replace the hard rank threshold with the entropy of the normalised spectrum:

$$\text{RankMe}(Z) = \exp\left(-\sum_k p_k \log p_k\right), \quad p_k = \frac{\sigma_k(Z)}{\|\sigma(Z)\|_1} + \epsilon, \quad (6.1)$$

with no centring and no mean subtraction; the paper does not give ϵ a numeric value in this equation.² Its selling point is that it tracks downstream accuracy well enough to select hyperparameters without labels, coming within “less than half a point” of a label oracle. Its limitation is stated by the authors and matters: high rank is “only a necessary condition”, and “RankMe should only be used to compare different runs of a given method.”

Alignment and uniformity — the two-axis view

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,y) \sim p_{\text{pos}}} \|f(x) - f(y)\|_2^\alpha, \quad \mathcal{L}_{\text{unif}} = \log \mathbb{E}_{x,y \sim p_{\text{data}}} e^{-t\|f(x)-f(y)\|_2^2}, \quad (6.2)$$

usually $\alpha = t = 2$, on ℓ_2 -normalised features.³ A collapsed encoder scores a *perfect* 0 on alignment and the worst possible 0 on uniformity. That pairing is the point: neither number alone is interpretable, and their trade-off is the axis good representations live on.

6.2 A real measurement

I ran the ablation the paper does not: a minimal JEPA on synthetic shape-classification data, 2,500 steps, identical in every respect except the target branch.⁴

target branch	final loss	emb. std	RankMe	probe acc.
untrained encoder (control)	—	0.165	16.75	31.1%
no stop-gradient	0.0037	0.0016	8.78	30.5%
stop-gradient only	9.86	0.163	15.57	38.6%
stop-gradient + EMA	9.17	0.150	15.57	42.1%

¹SimSiam §4.1 and Fig. 2: “with stop-gradient, the std value is near $\frac{1}{\sqrt{d}}$. This indicates that the outputs do not collapse, and they are scattered on the unit hypersphere.”

²Q. Garrido, R. Balestriero, L. Najman, Y. LeCun, *RankMe*, ICML 2023, Eqs. 1–2. <https://arxiv.org/abs/2210.02885> Practical notes from the paper: use 25,600 training embeddings; “10000 samples are enough to obtain more than 95% of the final rank”. Derived from Roy & Vetterli’s effective rank (2007). The 10^{-7} that circulates for ϵ belongs to the *classical* thresholded rank the paper contrasts RankMe with (“typically 10^{-7} for float32”), not to Eq. (2); our code uses it as a floor, which is a choice, not a citation.

³T. Wang, P. Isola, *Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere*, ICML 2020. <https://arxiv.org/abs/2005.10242> Their Theorem 1 shows both properties fall out of the $M \rightarrow \infty$ limit of the contrastive loss — iff for alignment, conditionally for uniformity — and a remark in §5 observes that no encoder can be both perfectly aligned and perfectly uniform on finite data.

⁴Pure-numpy JEPA with hand-derived gradients, mean-pooled context (no attention) so the gradients stay checkable. 6 classes = 3 shapes $\times$ 2 scales with colour randomised so colour is not a shortcut; 600 labelled images for the ridge probe; $D = 32$, so healthy std is $1/\sqrt{32} = 0.177$. Script: `exp/mini_jepa_numpy.py`.

Table 6.1. Chance is 16.7%. The no-stop-gradient run has a loss $2,700\times$ lower than the others and a representation no better than an untrained network.

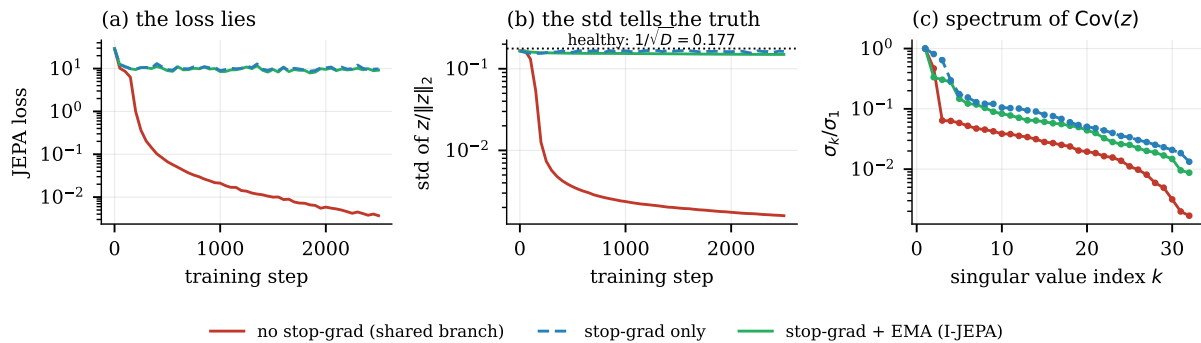


Figure 6.1. The same three runs over training. (a) The loss says the collapsed run is winning by three orders of magnitude. (b) The embedding std says it died at step ~ 400 . (c) Its spectrum decays fastest, and it loses 7 of 32 directions below 1% of the top singular value.

Three things to take from Table 6.1.

The loss is anti-correlated with quality here. Not uncorrelated — anti-correlated. This is the single most important practical fact in the book.

Stop-gradient alone is enough to avoid collapse. Exactly SimSiam’s finding, reproduced at $1/10\,000$ the scale.

EMA buys +3.5 points, not survival. 42.1% versus 38.6%. Also exactly what SimSiam reported: “The momentum encoder may be beneficial for accuracy, but it is not necessary for preventing collapsing.”

TRAP

Notice RankMe on the collapsed row: 8.78, out of a possible 32. Not catastrophic-looking. RankMe is scale-free — it sees the *shape* of the spectrum, not its magnitude — so a model whose embeddings have shrunk into a numerical pinhead can still report a respectable rank. The embedding std caught it immediately. Use both.

STOP AND THINK 6.1 The untrained control scores 31.1% and the collapsed run scores 30.5%. Suppose you had not run the control. Which of the four diagnostics would still have told you something was wrong, and which would you have misread as “fine, just early”?

EXERCISE 6.1 Run `python collapse_lab.py --steps 1200` on your own machine. It runs four variants: no stop-gradient, stop-gradient only, stop-gradient + EMA, and a *frozen* target ($m = 1$, predicting an untrained network’s features). Before you look: rank the four by final loss, then by probe accuracy, and predict whether the two orderings agree. Then plot with `plot_collapse.py` and see how close you were.

OPEN interactive/collapse-simulator.html trains the same four variants live in your browser in about ten seconds, with the diagnostics updating as it goes. Use it to build intuition, then use the PyTorch version to check that the intuition survives contact with a real ViT.

The architecture, precisely

Everything up to here was motivation. From here on the book is constructive: this chapter fixes notation and states the method exactly; the next four build it.

7.1 Notation

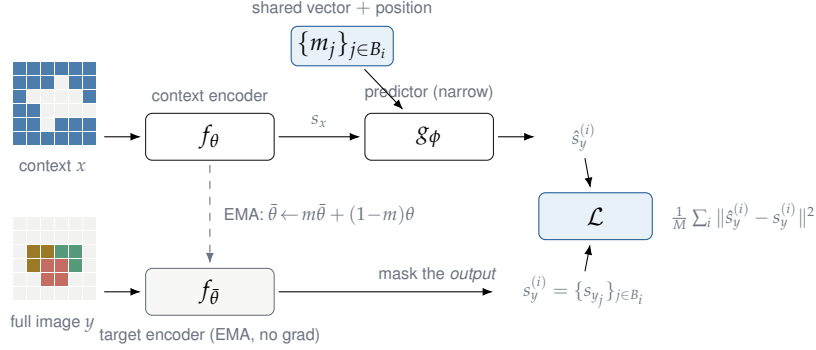


Figure 7.1. I-JEPA in full. One context block, $M = 4$ target blocks, one predictor applied M times. Three things to notice: the context encoder sees *only* the blue patches; the target encoder sees the *whole* image and is masked afterwards; and the only gradient path is $\mathcal{L} \rightarrow g_\phi \rightarrow f_\theta$.

Fix an image y , cut into N non-overlapping patches.

- The **target encoder** $f_{\bar{\theta}}$ processes the whole image, producing $s_y = \{s_{y_1}, \dots, s_{y_N}\}$.
- Sample M possibly-overlapping **target blocks**. Write B_i for the patch indices of block i , and $s_y^{(i)} = \{s_{y_j}\}_{j \in B_i}$.
- Sample one **context block** with index set B_x , delete from it every index appearing in any B_i , and encode the survivors with the **context encoder** f_θ to get $s_x = \{s_{x_j}\}_{j \in B_x}$.
- The **predictor** g_ϕ is run once per target block: $\hat{s}_y^{(i)} = g_\phi(s_x, \{m_j\}_{j \in B_i})$, where the mask tokens m_j are “a shared learnable vector with an added positional embedding”.

Mask the output, not the input

This is the sentence in the paper that carries the most weight per word: “Note that the target blocks are obtained by masking the *output* of the target-encoder, not the input. This distinction is crucial to ensure target representations of a high semantic level.” Because the target encoder attends over the whole image before masking, each s_{y_j} is *contextualised* — it encodes “the ear of a dog that is over there” rather than “a patch of brown fur”. The ablation is decisive:

masking applied to	arch.	epochs	IN-1% top-1
target-encoder <i>output</i>	ViT-H/16	300	67.3
target-encoder <i>input</i>	ViT-H/16	300	56.1

Table 7.1. 11.2 points from moving one masking operation past one encoder.¹

7.2 The loss, and the four-place disagreement

The paper writes the objective as an average squared ℓ_2 distance:

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^M D(\hat{s}_y^{(i)}, s_y^{(i)}) = \frac{1}{M} \sum_{i=1}^M \sum_{j \in B_i} \|\hat{s}_{y_j} - s_{y_j}\|_2^2. \quad (7.1)$$

Note the normalisation: it divides by M but not by $|B_i|$.

The released code does something else. Four differences between paper and implementation are worth knowing before you write a line, because three of them change results:

	what the paper says	what the code does
target normalisation	nothing at all	a <i>stateless</i> <code>F.layer_norm</code> over the feature dim, applied before masking
loss	squared ℓ_2 , Eq. (7.1)	<code>F.smooth_l1_loss</code> , i.e. Huber with $\beta = 1$, mean over all elements
weight decay	“linearly increased from 0.04 to 0.4”	a <i>cosine</i> ramp $0.04 \rightarrow 0.4$
LR warmup	$10^{-4} \rightarrow 10^{-3}$ over 15 epochs	<code>start_lr: 0.0002, warmup: 40</code>

Table 7.2. Paper versus `facebookresearch/ijepa` (archived Aug 2024, one commit). Neither is wrong; they simply are not the same experiment.²

The target LayerNorm is the one to internalise. It is unparameterised — no learnable scale or bias, just per-token standardisation — and it does two jobs. It fixes the scale of the targets even as the EMA encoder drifts, so the loss cannot be reduced by shrinking the targets. And it puts the residuals inside the quadratic regime of `smooth_l1`, which is why the Huber loss behaves like $\frac{1}{2}$ MSE in practice. In our code:

```
with torch.no_grad():
    h = target_encoder(imgs)                # [B, N, D] whole image
    h = F.layer_norm(h, (h.size(-1),))      # stateless: no weight, no bias
    targets = [apply_mask(h, m) for m in tgt_masks] # then mask
```

Key idea

The gradient graph of I-JEPA is a single path: $\mathcal{L} \rightarrow g_\phi \rightarrow f_\theta$. The target encoder is not in it. That asymmetry is the entire anti-collapse mechanism, and Chapter 5 is why it works.

STOP AND THINK 7.1 Suppose you deleted the target LayerNorm but kept everything else. Name a strategy the network could then use to reduce Eq. (7.1) without learning anything, and explain why the stop-gradient does *not* block it.

7.3 Sizes

encoder	pred. depth	pred. width	pred. heads	note
ViT-B/16	6	384	= encoder	
ViT-L/16	12	384	= encoder	encoder width is 1024
ViT-H/14	12	384	= encoder	the headline model
ViT-G/16	16	384	= encoder	IN-22K only
nano-cifar (ours)	3	96	3	encoder $192d \times 6$

Table 7.3. Predictor geometry. Width is *fixed* at 384 regardless of encoder size; only depth scales.³

Both effects have the same explanation. Depth gives the predictor the capacity to model a genuinely hard mapping. Width, past a point, lets it memorise the encoder’s exact output instead of learning the structure of it — so a bottleneck is a regulariser, and the paper says so: “Having a width bottleneck in the predictor improves the downstream performances.”

Masking is the method

If you change one thing about I-JEPA and get a different paper, it is the mask sampler. Nothing else in the design has ablations this violent.

8.1 The recipe

1. Sample $M = 4$ **target blocks**, possibly overlapping each other, with area scale drawn from $(0.15, 0.20)$ of the grid and aspect ratio from $(0.75, 1.50)$.
2. Sample **one context block**, area scale $(0.85, 1.00)$, unit aspect ratio.
3. **Delete** from the context every patch belonging to any target block.

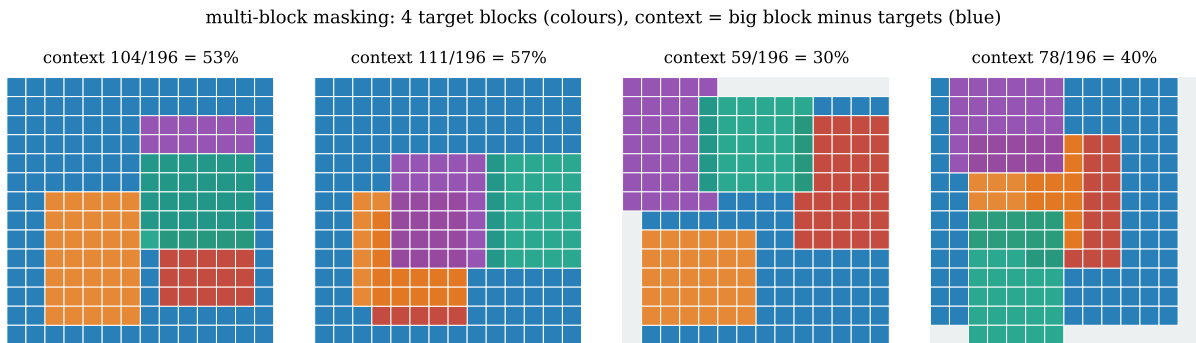


Figure 8.1. Four draws from the sampler on a 14×14 grid, using the paper’s parameters. Coloured cells are the four target blocks; blue is what survives in the context.

8.2 The ablation that justifies it

strategy	targets	M	avg. $ \text{ctx} /N$	IN-1%
multi-block	Block(0.15, 0.2)	4	0.25	54.2
block	Block(0.6)	1	0.40	20.2
random	Random(0.6)	1	0.40	17.6
rasterised	one quadrant $\rightarrow$ three	3	0.25	15.5

Table 8.1. ViT-B/16, 300 epochs, linear probe on 1% of ImageNet labels.¹

A 38.7-point spread between the best and worst row. For comparison, the entire representation-versus-pixel question in Chapter 2 was worth 26.2 points. Masking is not a detail — and the very next table contains an even larger effect.

Now the sensitivity curves, which are more instructive than the headline:

target scale		context scale		number of targets M	
range	top-1	range	top-1	M	top-1
(0.075, 0.2)	19.2	(0.40, 1.0)	31.2	1	9.0
(0.10, 0.2)	39.2	(0.65, 1.0)	47.1	2	22.0
(0.125, 0.2)	42.4	(0.75, 1.0)	49.3	3	48.5
(0.15, 0.2)	54.2	(0.85, 1.0)	54.2	4	54.2
(0.20, 0.25)	38.9				
(0.20, 0.3)	33.6				

Table 8.2. ViT-B/16, 300 epochs.²

Read Table 8.2 carefully, because it says something the abstract does not. Target scale is *non-monotone*: bigger targets help up to $(0.15, 0.2)$ and then hurt. The paper’s explanation is a single conditional clause:

predicting larger blocks improves accuracy “as long as the context is sufficiently informative”. Since the context is the complement of the targets, enlarging the targets starves the context. The default is a balance point, not an extreme.

The two requirements, and why they fight

Targets must be *large* so that filling them in cannot be done by copying adjacent texture — you have to know it is a tree. The context must be *informative and spatially spread* so that the prediction is possible at all. Both are drawn from one N -patch budget, so every point of one is taken from the other. $M = 4$ at scale $(0.15, 0.2)$ against a context of $(0.85, 1.0)$ is where the paper found the optimum.

$M = 1 \rightarrow 9.0\%$ is the most dramatic single number in the whole ablation suite. One target block per image is barely above chance. Four is 54.2. The reason is that each additional block is a nearly-free extra prediction problem sharing one encoder forward pass, so M is a signal-per-FLOP multiplier.

8.3 Batching, and where the 0.25 comes from

Both mask types have variable size, but tensors do not. Two mechanisms in the reference implementation fix this, and one of them explains a number in the paper.

Block shapes are drawn once per mini-batch. A seeded generator, shared across DataLoader workers, picks (h, w) for the targets and the context; every image in the batch gets identically-shaped blocks and only the *positions* vary.

Index lists are truncated to the batch minimum. The context is a set difference, so its size still varies per image. The collator slices every index list to the shortest one in the batch, in raster order.

That second mechanism has a consequence I only noticed by measuring it. Under the paper’s own parameters at $G = 14$, the *per-image* context ratio averages 0.409. After truncation to the batch minimum it drops, and it keeps dropping as the batch grows:

per-GPU batch	median n_{ctx}	range	effective ratio
32	57	31–78	0.291
64	53	18–74	0.270
128 (the paper’s)	51	25–78	0.260

The shipped config uses `batch_size: 128` per GPU, and at that batch size the measured effective ratio is 0.260 — within a hundredth of the “Avg. Ratio 0.25” reported in Table 8.1. That is suggestive rather than conclusive: Table 8.1 is a ViT-B/16 run ($G = 14$) whose per-GPU batch size the paper never states, and the config with `batch_size: 128` is the ViT-H/14 at 224², which has a 16×16 grid. What is *certain* is the mechanism — the truncation is right there in `multiblock.py` — and that the number you get depends on your batch size, which no paper reports next to it.³

TRAP

Truncation is a plain slice of a raster-ordered index list, not a random subsample — so it systematically discards the bottom-right of the region. It also means your *effective* context is smaller than your configured context scale, by an amount that depends on your batch size. If you change batch size and your results move, this is a candidate explanation.

Two upstream quirks

Worth knowing so you are not confused reading the original:

Scale and aspect ratio share one random draw. `_sample_block_size` calls `torch.rand` once and uses the result for both, so larger target blocks are always proportionally taller. Almost certainly unintended. Our `MaskConfig` draws them independently by default; set `couple_scale_and_ar=True` for bit-faithfulness. Empirically the correlation between block area and aspect ratio is 0.59 coupled versus 0.04 independent.

min_keep is a strict inequality (`len(mask) > min_keep`), and on repeated failure the sampler progressively drops constraints until a valid context exists rather than giving up.

EXERCISE 8.1 Run `ijepa.masks.context_ratio(cfg)` for grids $G \in \{8, 14, 16\}$ and confirm the numbers above. Then predict what happens to the effective context ratio as batch size $\rightarrow 1$, and check it.

OPEN `interactive/masking-visualizer.html` lets you drag the scale and aspect-ratio ranges and M , and shows the resulting context ratio live. Try to find a setting that reproduces the “block” and “rasterised” rows of Table 8.1, and one that gives a context ratio of 0.25 with $M = 8$.

³Measured over 200 batches per setting with my own torch-free mirror of the sampler, `exp/test_masks_numpy.py`, which also asserts that context and targets never overlap and that all indices are in range. The identical measurement in the browser (`interactive/masking-visualizer.html`) reproduces it to within 0.01.

Writing the encoder and the predictor

Now we build. This chapter is the two model classes; the next is the training step. Full sources are in `code/ijepa/vit.py`.

9.1 Positional embeddings, fixed

I-JEPA uses non-learned 2-D sinusoidal positions. Half the channels encode the row, half the column; within each half the layout is $[\sin(\text{pos} \cdot \omega) \mid \cos(\text{pos} \cdot \omega)]$ with $\omega_k = 10000^{-2k/(d/2)}$.

```
def sincos_2d(dim, grid):
    assert dim % 4 == 0
    def _1d(d, pos):
        omega = 1.0 / (10000.0 ** (torch.arange(d // 2).double() / (d / 2)))
        out = pos.reshape(-1).double()[ :, None] * omega[None]
        return torch.cat([out.sin(), out.cos()], dim=1)
    r = torch.arange(grid).double()
    row, col = torch.meshgrid(r, r, indexing="ij") # row-major, matches flatten
    return torch.cat([_1d(dim // 2, row), _1d(dim // 2, col)], dim=1).float()
```

Register it as a buffer with `requires_grad=False`, and set `persistent=False` so it never enters your checkpoint — it is reconstructible from `(dim, grid)`, and keeping it out means a checkpoint saved at one resolution does not silently fight you at another.

9.2 The encoder

Standard pre-norm ViT. The one non-obvious ordering is on line 4:

```
def forward(self, x, mask=None):
    x = self.patch_embed(x).flatten(2).transpose(1, 2) # [B,3,H,W] → [B,N,D]
    x = x + self.pos_embed # position BEFORE masking
    if mask is not None: # then keep only the context
        x = torch.gather(x, 1, mask.unsqueeze(-1).expand(-1, -1, self.dim))
    for blk in self.blocks:
        x = blk(x)
    return self.norm(x) # [B, n_ctx, D]
```

Adding position *before* gathering is what lets a context token retain its absolute location. Gathering *before* the blocks is what makes the encoder cheap: with a 25% context you pay roughly 1/16 of the attention cost.

Everything else is uncontroversial and none of it is in the paper — it all comes from the released code: `LayerNorm(eps=1e-6)` pre-norm, GELU, `mlp_ratio=4`, `qkv_bias=True`, `trunc_normal_(std=0.02)` init, a final `LayerNorm`, and zero dropout and zero drop-path in every shipped config. (One exception worth knowing if you scale up: `vit_giant` uses `mlp_ratio=48/11`, not 4.)

TRAP

The reference implementation applies a BEiT-style depth-dependent rescale (`fix_init_weight`, dividing residual-branch weights by $\sqrt{2 \cdot \text{layer_id}}$) in both constructors — and then `helper.init_model` loops over every module again and re-initialises all `nn.Linear` weights, silently undoing it. So the released checkpoints were trained *without* the rescale, despite the code appearing to apply it. We omit it and say so.

9.3 The predictor

This is the only genuinely fiddly class. It takes context tokens plus a set of positions to predict, and returns one vector per requested position.

```
def forward(self, ctx, ctx_mask, tgt_mask):
    B, n_ctx, _ = ctx.shape
    x = self.embed(ctx) # enc width → pred width (384)
    pe = self.pos_embed.expand(B, -1, -1) # predictor's OWN sincos table
    x = x + gather(pe, ctx_mask) # re-add position, in pred space

    tok = self.mask_token.expand(B, tgt_mask.size(1), -1) + gather(pe, tgt_mask)
    x = torch.cat([x, tok], dim=1) # [B, n_ctx + n_tgt, pred_dim]
    for blk in self.blocks: # FULL self-attention, no mask
        x = blk(x)
    x = self.norm(x)[ :, n_ctx:] # keep only the mask-token half
    return self.proj(x) # pred width → enc width
```

Five points that are easy to get wrong:

1. **The predictor has its own positional table**, at its own narrower width, and it re-adds position to the context tokens — on top of position the encoder already baked in. This is not a bug; it is what the reference does.
2. **One shared mask_token for every predicted position**. The positional add is the *only* thing distinguishing one mask query from another. If you dropped it, the predictor would output the same vector for every target patch, because attention is permutation-equivariant (Chapter 3).
3. **Full bidirectional self-attention over the concatenation**. No causal mask, no cross-attention, no block structure. Mask tokens attend to the context and to each other.
4. **Order is norm, then slice, then project**. `self.norm(x)[:, n_ctx:]` then `self.proj`. Slicing before the norm changes the statistics.
5. **Run the predictor M times**, once per target block, sharing one context forward pass. The paper says “we apply our predictor M times”; the released code achieves the same thing in one batched call with `x.repeat(len(masks), 1, 1)`, i.e. M independent sequences in one forward. Either way the blocks never attend to each other. Our loop is the readable form.

STOP AND THINK 9.1 Point 3 says mask tokens attend to each other. Consider two target blocks on opposite sides of the image, predicted in the *same* predictor call. Give one way this helps and one way it could let the model cheat. Then check: does I-JEPA actually put multiple blocks in one call, or one block per call? (Look at Figure 7.1 again.)

9.4 Confirm it works before you train it

```
$ python pretrain.py --smoke
device          cuda
preset          nano-cifar  grid 8x8 = 64 patches
params          encoder 2.68M predictor 0.37M
masking         M=4 targets (0.15, 0.2), context (0.85, 1.0), mean |ctx|/N = 0.34
SMOKE PASSED
  imgs          (256, 3, 32, 32)
  ctx_mask       (256, n_ctx)   (indices into 64)
  tgt_masks      4 x (256, n_tgt)
  target h       (256, 64, 192)
  context s_x     (256, n_ctx, 192)
  prediction     (256, n_tgt, 192) (must match target block slice)
```

The assertion runs *before* the banner prints, so a shape mismatch is a crash and not a cheerful message followed by a crash. Read the shape lines until they are obvious: `target h` spans all 64 patches while `context s_x` spans only n_{ctx} , and `prediction` matches `tgt_masks` exactly. The parameter counts are exact for this preset; n_{ctx} and n_{tgt} are drawn afresh each batch. Note that the printed `mean |ctx|/N` is the *per-image* average from `context_ratio()`; the batch-minimum figure your model actually sees is lower, as the next box explains.

TRAP

Your n_{ctx} will be smaller than you expect, and it shrinks as you raise the batch size. Measured on an 8×8 grid with the paper’s mask parameters: at batch 32 the batch-minimum context is a median of 13 of 64 patches (0.203); at batch 64, 12 (0.188); at batch 256, 11 (0.172).^a The paper’s 0.25 assumes a 14×14 grid. If you move to a small grid, either raise `context_scale` or lower the batch size — and know which one you changed.

^aMeasured with `exp/test_masks_numpy.py` over 200 batches per setting. This is the batch-minimum truncation of Chapter 8 biting harder on a coarse grid: with only 64 patches, one unlucky image drags the whole batch down.

EXERCISE 9.1 Break it deliberately, three ways, and predict the symptom before running: (a) move `x = x + self.pos_embed` to *after* the gather; (b) delete `+ gather(pe, tgt_mask)` from the mask tokens; (c) return `self.norm(x)[:, :n_ctx]` instead of `[:, n_ctx:]`. One of the three still trains to a low loss. Which, and what does that tell you about reading loss curves?

OPEN [interactive/vit-explorer.html](https://interactive.vit-explorer.html) steps through patch embedding, the sin-cos table, and attention maps one operation at a time.

The training step

The method is twenty lines. This chapter is those twenty lines, plus the schedules around them, plus the reasons.

10.1 Twenty lines

```
def jepa_step(encoder, predictor, target_encoder, imgs, ctx_mask, tgt_masks):
    # 1. targets: whole image through the EMA encoder, normalise, THEN mask
    with torch.no_grad():
        h = target_encoder(imgs)                # [B, N, D]
        h = F.layer_norm(h, (h.size(-1),))      # stateless LN (code, not paper)
        targets = [apply_mask(h, m) for m in tgt_masks]

    # 2. context: only the visible patches are ever encoded
    s_x = encoder(imgs, ctx_mask)               # [B, n_ctx, D]

    # 3. one prediction per target block, conditioned on its positions
    loss = imgs.new_zeros(())
    for m, t in zip(tgt_masks, targets):
        loss = loss + F.smooth_l1_loss(predictor(s_x, ctx_mask, m), t)
    return loss / len(tgt_masks)
```

and, per step, in this order:

```
set_lr_wd(opt, warmup_cosine(step, total, warmup, lr), cosine_ramp(step, total, 0.04, 0.4))
loss = jepa_step(...)
opt.zero_grad(set_to_none=True); loss.backward(); opt.step()
ema_update(encoder, target_encoder, linear_ramp(step, total, 0.996, 1.0)) # AFTER opt.step
```

10.2 The stop-gradient, three times over

It is worth being paranoid here, because a leak is silent and fatal. Our implementation blocks the gradient in three independent ways:

1. `torch.no_grad()` wraps the entire target branch, including the LayerNorm and the masking, so no graph node is ever created.
2. `make_target_encoder` sets `requires_grad=False` on every target parameter.
3. `param_groups` only walks encoder and predictor, so target parameters are not in the optimiser at all.

Any one would do. All three means that if you later refactor, you have to make the same mistake three times.

10.3 EMA

```
@torch.no_grad()
def ema_update(encoder, target_encoder, m):
    for pq, pk in zip(encoder.parameters(), target_encoder.parameters()):
        pk.mul_(m).add_(pq.detach(), alpha=1.0 - m)
```

Exact at both endpoints: $m = 0$ is a hard copy, $m = 1$ is a freeze. I-JEPA ramps m linearly from 0.996 to 1.0 across the whole run, so the target encoder is a slow-moving average early and effectively frozen at the end. Note what Chapter 5 says about that direction: small τ early keeps the predictor aligned; growing τ raises the target for p_j . The schedule is the automatic curriculum.

Two implementation notes. Only `parameters()` are averaged, not buffers — irrelevant here (no Batch-Norm) but a trap if you add any. And the update must come *after* `opt.step()`, or the target chases a stale online network.

10.4 Schedules, all per-iteration

quantity	paper (ViT-H/14, IN-1K)	ours (nano-cifar)
optimiser	AdamW	AdamW
batch size	2048 (128/GPU $\times$ 16)	256
LR	warmup $10^{-4} \rightarrow 10^{-3}$, cosine to 10^{-6}	warmup to 1.5×10^{-3} , cosine
warmup	15 epochs (paper) / 40 (config)	15% of steps
weight decay	0.04 $\rightarrow$ 0.4	0.04 $\rightarrow$ 0.4
EMA m	0.996 $\rightarrow$ 1.0 linear	0.996 $\rightarrow$ 1.0 linear
epochs	300	60
precision	bf16 (config; unmentioned in paper)	bf16 on CUDA, fp32 on MPS
grad clipping	none	none (<code>--clip</code> to enable)
augmentation	<code>RandomResizedCrop(scale=(0.3, 1.0))</code> only	same

Table 10.1. Hyperparameters. Everything in the left column comes from Appendix A.1 or the shipped YAML; the paper contains no hyperparameter table.¹

Two remarks on that table. The “no augmentation” claim is literally true of the config — horizontal flip, colour jitter and Gaussian blur are all implemented upstream and all switched off — with the single exception of `RandomResizedCrop`, which remains. And weight decay excludes biases and 1-D parameters, standard for ViTs.

TRAP

`mask_token` has shape $[1, 1, d]$, so the usual “exclude parameters with `ndim <= 1`” rule does *not* catch it, and the reference implementation therefore weight-decays it. Over a long run at `wd  $\rightarrow$  0.4` that drives towards zero the one parameter distinguishing a mask query from a bare positional embedding. We exclude it by name and flag the difference in `engine.param_groups`.

10.5 Running it

Each epoch prints one line:

```
ep <n> loss <L> std/(1/sqrt D) <s> rankme <r> eff_rank <k>/192 [verdict]
```

Read it right to left, not left to right.

- `std/(1/sqrt D)` is the collapse alarm. Healthy is 0.7–1.0; below 0.30 stop and reread Chapter 6.
- `eff_rank` is how many of the 192 embedding directions still carry signal. A steady slow decline is normal — the encoder is specialising. A cliff is dimensional collapse.
- `rankme` should track `eff_rank`. If they diverge, trust `eff_rank` and reread the trap box in §6.2.
- `loss` is the least informative number on the line. It will fall whether or not anything good is happening.

I am deliberately not printing my own numbers here. You have the hardware; the point of the chapter is that you learn to read the diagnostics rather than compare against a target I got on different silicon.

EXERCISE 10.1 Train once with the defaults, then twice more changing exactly one thing: `--loss l2` (the paper’s equation) and `--ema-start 1.0` (a frozen target, i.e. predicting an untrained network’s features). Record final probe accuracy for all three. Then predict, before running, what `--ema-start 0.0` does, and why Chapter 5 says it will not collapse.

Evaluating without fooling yourself

Self-supervised learning has no validation loss you can trust. This chapter is about not deceiving yourself, which in this field is most of the skill.

11.1 Which encoder, and how to pool

The paper is unambiguous and it is not the obvious choice: evaluate the *target* encoder, not the online one, and average-pool its patch tokens. There is no [CLS] token to read.

11.2 Three numbers, because any one of them lies

Linear probe. Freeze the encoder, extract features once, fit one linear layer. This is the standard number, and the reason it is standard is that a linear classifier cannot manufacture structure that is not already there.

kNN. No learned parameters at all, so it cannot paper over a mediocre representation with a well-tuned head. If your linear probe is much better than your kNN, the head is doing more work than the encoder.

The random-initialisation control. Same architecture, no training. This is the one people skip and it is the one that matters most, because a randomly initialised ViT is a surprisingly strong feature extractor on small images. Our `probe.py` prints it as a row and prints the delta:

	linear	kNN	
chance	10.00%	10.00%	
random init	<a>		← same architecture, no training
I-JEPA	<c>	<d>	
delta	<c-a>	<d-b>	← THIS is your result

The only claim worth making

“My I-JEPA reached X%” means nothing without the random-init row. The claim is the *delta*. Cite it that way and you will never be embarrassed by a reviewer.

11.3 Low-shot, and why the paper leads with it

Most of I-JEPA’s ablations report ImageNet-1% rather than the full linear probe — roughly 12 or 13 labelled images per class. That is deliberate. With all 1.28 M labels a linear probe can compensate for a lot of representational mush; with 13 images per class it cannot. Low-shot is a sharper instrument for comparing representations, and it is what you should use for your own ablations: `python probe.py --ckpt ... --frac 0.01`.

11.4 The published numbers, for calibration

method	arch.	ep.	IN linear	IN-1%
<i>no view augmentations</i>				
MAE	ViT-H/14	1600	77.2	71.5
data2vec	ViT-L/16	1600	77.3	73.3
CAE	ViT-L/16	1600	78.1	—
I-JEPA	ViT-B/16	600	72.9	—
I-JEPA	ViT-H/14	300	79.3	73.3
I-JEPA	ViT-H/16 ₄₄₈	300	81.1	77.3
<i>with view augmentations</i>				
DINO	ViT-B/8	300	80.1	70.0
iBOT	ViT-L/16	250	81.0	—
MSN	ViT-B/4	300	—	75.7

Table 11.1. ImageNet-1K linear evaluation and 1% semi-supervised top-1.¹

And the transfer picture, which is where the honest reading lives:

method	arch.	semantic			local	
		CIFAR100	Places205	iNat18	Clevr/Count	Clevr/Dist
data2vec	ViT-L/16	81.6	54.6	28.1	85.3	71.3
MAE	ViT-H/14	77.3	55.0	32.9	90.5	72.4
I-JEPA	ViT-H/14	87.5	58.4	47.6	86.7	72.4
DINO (<i>aug</i>)	ViT-B/8	84.9	57.9	55.9	86.6	53.4
iBOT (<i>aug</i>)	ViT-L/16	88.3	60.4	57.3	85.7	62.8

Table 11.2. Frozen-backbone linear transfer.²**TRAP**

Table 11.2 is where a careful reader learns something the abstract does not say. I-JEPA beats the augmentation-based methods decisively on Clevr/Dist (72.4 vs 53.4 and 62.8) — view invariance destroys depth information, exactly as Chapter 1 predicted. But it *loses* to iBOT on all three semantic columns, and loses badly on iNat18: 47.6 versus 57.3, a 9.7-point gap. On full-ImageNet fine-tuning it is 87.1 versus MAE’s 87.8. The paper’s own language on *transfer* is “decreases the gap” (Table 3 caption), not parity — though on ImageNet itself it does claim to “match the performance of view-invariant approaches such as iBOT” (§5) and to “surpass previous methods” (Table 2 caption). Keep the two claims apart.

EXERCISE 11.1 Reproduce the shape of Table 11.2 at your own scale. Train one encoder on CIFAR-10, then probe it on (a) CIFAR-10 classification and (b) a task you construct that needs *position* — for instance, predicting which quadrant contains the brightest patch. Compare against the random-init control on both. Which task shows the bigger delta, and does that match the pattern in the table?

V-JEPA: the same recipe on video

Video is where the argument for latent prediction gets sharper, because the unpredictable detail is now unpredictable in *time* as well as space. If you have understood Chapters 7–10, V-JEPA is four changes.¹

1. Tubelets instead of patches

A token is now “a 16×16 pixel block spanning 2 consecutive frames”. Concretely: sample 16 frames at temporal stride 4 (covering 64 raw frames, which Appendix B calls “roughly 2 seconds” at 30 fps while §3.4 says “roughly 3 second clips on average” — the paper is inconsistent), resize to 224×224 , apply a 3-D convolution with $2 \times 16 \times 16$ kernel and matching stride. That gives $8 \times 14 \times 14 = 1568$ tokens, with absolute 3-D sin-cos positions and no [CLS].

2. 3-D multi-block masking, and two masks per clip

Spatial blocks are sampled and then *extended through the entire temporal extent* — so a mask is a tube, not a blob. Two mask types are used simultaneously on every clip:

mask	blocks	spatial coverage each	aspect ratio
short-range	8	15% of the frame	(0.75, 1.5)
long-range	2	70% of the frame	(0.75, 1.5)

The union gives what the paper calls “an average masking ratio of $\sim 90\%$ ”. Because both masks are applied to the same clip, the y -branch is computed once and shared — “we only need to compute the y -representation once.”

Why tubes and not per-frame blobs? Because if you mask a different region in each frame, the model can interpolate temporally from adjacent frames and never has to learn motion. A tube forces genuine prediction across time.

3. ℓ_1 instead of squared ℓ_2

Stated plainly: “we modify it to use an ℓ_1 regression, which we found to be more stable.” The target normalisation is the same stateless `F.layer_norm` as I-JEPA. Targets are again taken from the *output* of the y -encoder: “We apply masking at the input of the x -encoder, and at the output of the y -encoder to construct contextualized targets.”

4. Everything else is the same

EMA target encoder with stop-gradient, momentum $0.998 \rightarrow 1.0$ linear (I-JEPA used 0.996); no decoder anywhere in the pretraining path; predictor is again a narrow transformer, 12 blocks at width 384. Gradient clipping at 10.0 is added after warmup — I-JEPA had none.

12.1 Does it work

model	params	K400	SSv2	IN1K	Places205
V-JEPA ViT-L/16	200M	80.8	69.5	74.8	60.3
V-JEPA ViT-H/16	630M	82.0	71.4	75.9	61.7
V-JEPA ViT-H/16 ₃₈₄	630M	81.9	72.2	77.4	62.8

Table 12.1. Frozen backbone, attentive probe.²

And the controlled feature-versus-pixel test, the video analogue of Table 2.1: ViT-L/16, same data, same 90k iterations — pixels give 68.6 / 66.0 / 73.3 on K400 / SSv2 / IN1K, features give **73.7 / 66.2 / 74.8**. A 5.1-point gain on K400, and essentially none on SSv2. The argument for latent targets is real but not uniform.

Training data: *VideoMix2M*, “approximately 2 million videos” assembled from HowTo100M, Kinetics-400/600/700 and Something-Something-v2 with validation overlap removed. The constituent datasets are public; the assembled index is not distributed with the code, so reproducing the exact mix is left to you.

¹ A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, N. Ballas, *Revisiting Feature Prediction for Learning Visual Representations from Video*, 2024. <https://arxiv.org/abs/2404.08471> Code: <https://github.com/facebookresearch/jepa>

TRAP

The configs say `epochs: 300` and this does *not* mean 300 passes over the data — it means $300 \times \text{ipe} = 300 \times 300 = 90,000$ *iterations*. Also note that Table 6 lists ViT-L/16 at 200M parameters, where a standard ViT-L/16 is $\approx 300\text{M}$; the same column lists a baseline the same way, so it is a consistent in-paper convention rather than a typo, but do not quote it as a general fact.

STOP AND THINK 12.1 V-JEPA masks tubes so the model cannot interpolate across frames. What is the analogous shortcut in *image* I-JEPA, and which specific ablation row in Table 8.1 is evidence that it exists?

V-JEPA 2, and the predictor as a world model

This is where the 2022 position paper’s actual point arrives. A JEPA predictor maps (state embedding, conditioning) to (next state embedding). Replace the conditioning — “which patch” — with an *action*, and you have a model an agent can plan with.¹

13.1 Stage 1: scale the action-free model

Same objective, much more data and a resolution curriculum.

quantity	V-JEPA (2024)	V-JEPA 2 (2025)
pretraining samples	2M videos	22M (<i>VideoMix22M</i>)
video hours	—	>1 million
largest encoder	630M (ViT-H)	1B (ViT-g)
positional encoding	absolute 3-D sin-cos	3-D RoPE
iterations	90,000	252,000 (12k warmup, 228k main, 12k cooldown)

Table 13.1. ²

The three-phase schedule is the practical trick: train mostly at 16 frames and 256², then in a short cooldown raise duration and resolution to 64 frames at 384² while decaying the learning rate. They report this as an “8.4× reduction in GPU time” versus training at the final resolution throughout, and note the ceiling: “we did not observe any further improvement beyond 64 frames.”

13.2 Stage 2: condition the predictor on actions

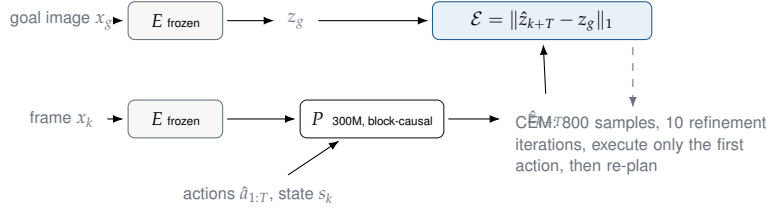


Figure 13.1. V-JEPA 2-AC. The encoder is frozen and applied per frame; only the action-conditioned predictor is trained, on under 62 hours of *unlabelled* robot video. Planning is receding-horizon control against a goal-image embedding.

The post-training is remarkably cheap given what it produces:

- **Data:** “approximately 62 hours of unlabeled video from the raw Droid dataset”, 23k trajectories of a 7-DoF Franka arm. Unlabelled means no reward, no task label, no success flag — “we only use the raw video and end-effector state signals”.
- **Frozen encoder.** “the encoder is kept frozen during this post-training phase”. Only the predictor learns.
- **Actions** are the 7-D delta of end-effector state (3 position, 3 Euler, 1 gripper) between adjacent frames. The predictor is 24 layers, 16 heads, width 1024, with block-causal attention so each patch at time t sees the action, pose and patches at t and all earlier steps.
- **Loss:** ℓ_1 again, as teacher-forcing plus a rollout term, differentiating through only one recurrent step ($T = 2$).

Planning without a reward function

This is the punchline of the whole research programme, and it is one equation:

$$\mathcal{E}(\hat{a}_{1:T}; z_k, s_k, z_g) = \|P(\hat{a}_{1:T}; s_k, z_k) - z_g\|_1, \quad a^* = \arg \min_{\hat{a}_{1:T}} \mathcal{E}. \quad (13.1)$$

Encode the goal image. Roll candidate action sequences through the predictor. Score each by ℓ_1 distance to the goal embedding. Execute the first action of the best sequence, observe, re-plan. Minimisation is by the Cross-Entropy Method: sample, refit a Gaussian to the top- k , iterate. No reward function, no value function, no policy network.

¹M. Assran et al., *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*, 2025. <https://arxiv.org/abs/2506.09985>
Code: <https://github.com/facebookresearch/vjepa2>

method	Reach	Grasp cup	Grasp box	Reach w/cup	Reach w/box	P&P cup	P&P box
Octo	100%	15%	0%	15%	70%	15%	10%
V-JEPA 2-AC	100%	65%	25%	75%	75%	80%	65%

Table 13.2. Zero-shot on Franka arms in two labs, neither present in Droid, with an uncalibrated monocular RGB camera, 10 trials per task.³

Two honest caveats the table hides. Pick-and-place uses two intermediate *sub-goal* images, not a single final goal. And the energy landscape, while “relatively smooth and locally convex”, has its minimum at roughly $(0, -0.05)$ where ground truth is $(0, -0.10)$ — a systematic $2\times$ underestimate of displacement.

13.3 What the authors say does not work yet

Read this section of any paper first. V-JEPA 2’s limitations are specific and they define the research frontier:

- **Camera pose sensitivity.** The model must infer the action coordinate frame from monocular RGB, and often the robot base is not even in frame, so “the problem of inferring the action coordinate axis is not well defined”. Their fix was manual: “we manually tried different camera positions before settling on one that worked well”.
- **Long-horizon planning fails, for two compounding reasons.** Autoregressive rollouts accumulate error, *and* the search space grows exponentially in the horizon. This is why pick-and-place needs sub-goals. Their proposed remedy is hierarchical models across spatial and temporal scales.
- **Goals must be images.** “it may be more natural to express goals in other forms, such as with language.”
- **Scale is untested.** “in this work we scaled V-JEPA 2 models up to a modest 1B parameters ... Additional work is needed ... to develop scalable pre-training recipes that lead to sustained performance improvements with scale.” For a research programme premised on scalability, that is a significant open question.

The arc, in one sentence

I-JEPA showed latent prediction gives good image representations; V-JEPA showed the recipe transfers to video with only a loss and a masking change; V-JEPA 2 showed that swapping position-conditioning for action-conditioning turns the same predictor into a planner. Nothing architectural changed across the three.

EXERCISE 13.1 Build the toy version. interactive/latent-planning.html implements exactly the loop above in a 2-D gridworld: an encoder, an action-conditioned predictor, and CEM search against a goal embedding. Do three things with it. (1) Shrink the planning horizon to 1 and find a layout where the greedy plan fails — you have reproduced the sub-goal problem. (2) Add noise to the predictor and watch error accumulation destroy long rollouts. (3) Replace the ℓ_1 cost with ℓ_2 and see whether the argmin moves.

OPEN interactive/latent-planning.html

What is genuinely unsettled

A curriculum that ends with “and now you know how it works” has taught you the wrong thing. Here is where the ground is soft, which is also where the research projects are.

1. The anti-collapse mechanism has no proof at the scale it is used

Chapter 5’s analysis is rigorous and it is *linear*. Result 1 (no stop-gradient $\Rightarrow$ collapse) is a theorem. Result 2 (with stop-gradient, collapse is a saddle) holds under three assumptions — $W_a = \tau W$, isotropic data, symmetric W_p — none of which hold for a ViT-H on ImageNet. SimSiam’s own paper is admirably direct that non-collapse “still remains as an empirical observation”. And I-JEPA runs *no* EMA ablation and *no* stop-gradient ablation; its “proven essential” claim is a citation to DINO, MoCo v3 and iBOT, not a measurement of its own.¹

That is a gap you could work in.

2. The masking parameters look tuned, and nobody knows the rule

Table 8.2 shows sharp non-monotone sensitivity around (0.15, 0.20) with $M = 4$ on a 14×14 grid at 224^2 . There is no theory predicting those values, no reported dependence on grid size, and, as Chapter 8’s measurement shows, the effective context ratio silently depends on batch size. Whether “4 blocks at 15–20%” is a fact about images or a fact about 196 patches at batch 2048 is, as far as I can tell, unanswered.

3. Semantic transfer is still behind

Read Table 11.2 again without the abstract’s framing. I-JEPA loses to iBOT on CIFAR100, Places205 and iNat18, and by 9.7 points on the last. It loses to MAE on Clevr/Count. It loses to MAE on full-ImageNet fine-tuning, 87.1 versus 87.8. Its genuine wins are efficiency and depth-like local tasks. Both matter; neither is “latent prediction dominates”.

4. LeCun’s own criteria are not what I-JEPA implements

This is the most interesting tension in the literature and it is almost never mentioned. The position paper lists four criteria for training a JEPA: maximise the information in s_x about x , maximise the information in s_y about y , make s_y predictable from s_x , minimise the information in z . Criteria 1 and 2 are what “prevent the energy surface from becoming flat by informational collapse”, and the recommended instantiations are VICReg and Barlow Twins.²

I-JEPA does none of that. It uses the asymmetric EMA trick — the family LeCun explicitly *excludes* from the class he advocates. So the flagship JEPA implementation prevents collapse by a mechanism its own conceptual author argues against. A well-posed research question falls straight out: what does I-JEPA look like with a VICReg-style variance-covariance regulariser instead of, or alongside, the EMA? V-JEPA’s code contains a variance regulariser (`reg_fn`) that is disabled in every shipped config (`reg_coeff: 0.0`), which suggests someone tried.

5. Five projects you could actually do

1. **Replace EMA with VICReg terms** in `jepa_step` and compare on CIFAR-10 with matched compute. Chapter 6’s diagnostics are your measurement apparatus. This is question 4 above, at a scale you own.
2. **Map the masking rule across grid sizes.** Sweep target scale and M at $G \in \{8, 14, 16\}$, holding the *effective* (batch-truncated) context ratio fixed rather than the configured one. Does the optimum move?
3. **Test Chapter 5’s predictions on a real ViT.** The theory says a symmetric predictor needs EMA and that predictor-only weight decay can substitute for it. Both are one-line changes in `engine.py`.
4. **Predictor depth versus width at small scale.** The paper reports +2.9 for depth and –2.3 for width at ViT-L. Does the sign survive at 3M parameters, and is there a ratio that predicts it?
5. **JEPA on a modality with no augmentation story** — accelerometer, ECG, spectrograms. This is the strongest argument for the method and the least tested. The masking recipe is the open question: what is a “semantic block” in a 1-D signal?

¹I-JEPA §3, *Loss*, cites [18, 25, 79] = DINO, MoCo v3 (Chen, Xie & He, *An Empirical Study of Training Self-Supervised Vision Transformers*) and iBOT.

²LeCun 2022, §4.5, p. 26 and Fig. 13 caption: “Examples of such non-contrastive criteria for JEPA training include VICReg and Barlow Twins.”

Answers, and where to go next

15.1 Answers to the STOP AND THINK boxes

1.1 In (b) the decoder must produce *pixels*, which are given data — it cannot make the target easier. In (c) the target is produced by a *learned* encoder, so the system can reduce the loss by making s_y constant. The conditioning z is identical; what changed is that the target became a free variable.

2.1 The generative architecture’s target has fixed informational capacity — pixels are pixels. The JEPA’s target has *learnable* capacity, and the cheapest way to satisfy any predictor is to drive that capacity to zero. “Informational capacity of z is low” protects (b) because the only free thing is the decoder; in (c) the encoder is free too.

3.1 The same vector for every position. Attention is permutation-equivariant, so with identical inputs at every masked slot the outputs must be identical up to the influence of the context, which is shared. The positional embedding is the only symmetry-breaking signal.

4.1 *Depth* — more precisely, the implicit regularisation of gradient descent in deep networks. The singular-value dynamics are cubic, so small directions grow much more slowly than large ones. Neither of Corollary 1’s remedies addresses it, because it is not about the minimiser set at all; it is about the path. That is why explicit spectral regularisers (VICReg’s covariance term, Barlow Twins’ off-diagonal term) exist as a third option.

5.1 On the collapsed branch $s_j = 0$, so $\dot{p}_j = -\eta p_j$: with $\eta = 0$ the predictor does not move at all, and the branch is neutrally stable in p and unstable in s — a saddle. A saddle is escaped by almost every initialisation, which is why training works; but “almost every” is not “every”, and with weight decay Result 3 shows the exceptional set is a genuine open region, not a measure-zero curve.

6.1 The embedding std would have caught it: 0.0016 against a healthy 0.177 is a factor of 100, not a slow start. The spectrum would too. RankMe at 8.78 out of 32 is the one you could have talked yourself into as “fine, still warming up”, and the loss — $2,700\times$ lower than the healthy runs — would have actively misled you.

7.1 Without the target LayerNorm, the target encoder’s output scale is free. The EMA slowly copies the online encoder, so if the online encoder shrinks its outputs the targets shrink with it, and $\|\hat{s}_y - s_y\|^2 \rightarrow 0$ by pure scale reduction. The stop-gradient does not block this because the online encoder is being updated through the *context* branch, which shares its weights — the leak is through the EMA, not through the loss graph. The LayerNorm removes scale from the target entirely.

This is checkable and I checked it. Running the browser simulator’s no-stop-gradient variant for 4000 steps: with the target LayerNorm *on*, the mean embedding norm $\|z\|$ stays at 9.2 and collapse shows up as every embedding pointing the same way (std ratio 0.011). With it *off*, $\|z\|$ falls to 0.051 while the std ratio reads a deceptively healthy 0.189. Same loss, same architecture, two entirely different degeneracies — and the std-ratio diagnostic only catches one of them. Which is the real argument for keeping several diagnostics rather than one.

9.1 It helps because two blocks can share reasoning about global scene layout in a single pass. It could cheat if one block’s content were inferable from the other’s *answer* rather than from the context — but since the mask tokens carry no content, only position, there is nothing to leak. And in fact I-JEPA gives each target block its own predictor sequence, so blocks never see each other at all. V-JEPA does the same for its two mask types — “we need to forward propagate the x -encoder and predictor separately for each mask, we only need to compute the y -representation once” — so what it amortises is the target pass, not the prediction.

12.1 The shortcut is copying adjacent texture — filling a masked patch from its neighbours without any semantic understanding. The evidence is the “random” row of Table 8.1: scattered single patches, 60% masked, score 17.6 against multi-block’s 54.2, because every masked patch has an unmasked neighbour. Large contiguous blocks remove the neighbour.

15.2 The reading order I would use

If you read five papers after this book, read them in this order.

1. **SimSiam** (Chen & He, 2021) — the cleanest possible experiment on what actually prevents collapse. Short, and every ablation earns its place. <https://arxiv.org/abs/2011.10566>
2. **Tian, Chen & Ganguli** (2021) — Chapter 5’s source. Read §2 and §3 with pen and paper; the rest is

DirectPred, which is a bonus. <https://arxiv.org/abs/2102.06810>

3. **I-JEPA** (Assran et al., 2023) — now read it properly, with the ablation tables first. <https://arxiv.org/abs/2301.08243>
4. **VICReg** (Bardes, Ponce & LeCun, 2022) — the road not taken, and the one LeCun advocates. <https://arxiv.org/abs/2105.04906>
5. **V-JEPA 2** (Assran et al., 2025) — read §3.2 for the planning loop, §4.2 for the robot results, and §4.3 for the limitations (§9 adds the hierarchical-models remedy and the scaling caveat). <https://arxiv.org/abs/2506.09985>

The LeCun position paper is worth reading once, in full, for the framing rather than the technique: <https://openreview.net/pdf?id=BZ5a1r-kVsf>. §4 is the relevant part.

15.3 Every source cited in this book

Primary. Assran, Duval, Misra, Bojanowski, Vincent, Rabbat, LeCun, Ballas. *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)*, ICCV 2023. <https://arxiv.org/abs/2301.08243> · **code (archived)** <https://github.com/facebookresearch/ijepa>

LeCun. *A Path Towards Autonomous Machine Intelligence*, v0.9.2, 2022. <https://openreview.net/pdf?id=BZ5a1r-kVsf>

Bardes, Garrido, Ponce, Chen, Rabbat, LeCun, Assran, Ballas. *Revisiting Feature Prediction for Learning Visual Representations from Video (V-JEPA)*, 2024. <https://arxiv.org/abs/2404.08471> · <https://github.com/facebookresearch/jepa>

Assran et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*, 2025. <https://arxiv.org/abs/2506.09985> · <https://github.com/facebookresearch/vjepa2>

Collapse: mechanism and theory. Tian, Chen, Ganguli. *Understanding self-supervised learning dynamics without contrastive pairs*, ICML 2021. <https://arxiv.org/abs/2102.06810>

Jing, Vincent, LeCun, Tian. *Understanding Dimensional Collapse in Contrastive Self-supervised Learning*, ICLR 2022. <https://arxiv.org/abs/2110.09348>

Grill et al. *Bootstrap Your Own Latent (BYOL)*, NeurIPS 2020. <https://arxiv.org/abs/2006.07733>

Chen, He. *Exploring Simple Siamese Representation Learning (SimSiam)*, CVPR 2021. <https://arxiv.org/abs/2011.10566>

Richemond, Grill, Althé, Tallec, Strub, Brock, Smith, De, Pascanu, Piot, Valko. *BYOL works even without batch statistics*, 2020. <https://arxiv.org/abs/2010.10241>

Collapse: the other three escapes. Chen, Kornblith, Norouzi, Hinton. *SimCLR*, ICML 2020. <https://arxiv.org/abs/2002.05709>

He, Fan, Wu, Xie, Girshick. *Momentum Contrast (MoCo)*, CVPR 2020. <https://arxiv.org/abs/1911.05722>

Zbontar, Jing, Misra, LeCun, Deny. *Barlow Twins*, ICML 2021. <https://arxiv.org/abs/2103.03230>

Bardes, Ponce, LeCun. *VICReg*, ICLR 2022. <https://arxiv.org/abs/2105.04906>

Caron, Misra, Mairal, Goyal, Bojanowski, Joulin. *SwAV*, NeurIPS 2020. <https://arxiv.org/abs/2006.09882>

Caron, Touvron, Misra, Jégou, Mairal, Bojanowski, Joulin. *Emerging Properties in Self-Supervised Vision Transformers (DINO)*, ICCV 2021. <https://arxiv.org/abs/2104.14294>

Assran et al. *Masked Siamese Networks (MSN)*, ECCV 2022. <https://arxiv.org/abs/2204.07141>

Diagnostics. Garrido, Balestriero, Najman, LeCun. *RankMe*, ICML 2023. <https://arxiv.org/abs/2210.02885>

Wang, Isola. *Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere*, ICML 2020. <https://arxiv.org/abs/2005.10242>

Roy, Vetterli. *The effective rank: a measure of effective dimensionality*, EUSIPCO 2007.

Background. Dosovitskiy et al. *An Image is Worth 16x16 Words (ViT)*, ICLR 2021. <https://arxiv.org/abs/2010.11929>

He, Chen, Xie, Li, Dollár, Girshick. *Masked Autoencoders Are Scalable Vision Learners (MAE)*, CVPR 2022. <https://arxiv.org/abs/2111.06377>

LeCun, Chopra, Hadsell, Ranzato, Huang. *A Tutorial on Energy-Based Learning*, 2006.

Vaswani et al. *Attention Is All You Need*, NeurIPS 2017. <https://arxiv.org/abs/1706.03762>

15.4 What is in the folder

JEPA_from_scratch.pdf	this book
code/	the PyTorch implementation; start with README.md, then pretrain.py --smoke
interactive/index.html	landing page for the four interactive pages
interactive/masking-visualizer.html	Ch. 8: drag the mask parameters, watch the context ratio
interactive/collapse-simulator.html	Ch. 4–6: train four variants live, watch three of them survive
interactive/vit-explorer.html	Ch. 3, 9: patch embedding, sin-cos positions, attention, step by step
interactive/latent-planning.html	Ch. 13: CEM planning against a goal embedding in a gridworld
exp/	the numpy experiments behind Figures 5.1, 5.2 and Table 6.1, and the mask-sampler test suite

Reference card

The method. With image y cut into N patches:

1. $h = \text{LN}(f_{\bar{\theta}}(y))$ over the *whole* grid, under `no_grad`; then gather target blocks $s_y^{(i)}, i = 1..M$.
2. $s_x = f_{\theta}(y, B_x)$ over the *context only*.
3. $\hat{s}_y^{(i)} = g_{\phi}(s_x, \{m_j\}_{j \in B_i})$ for each i .
4. $\mathcal{L} = \frac{1}{M} \sum_i \text{dist}(\hat{s}_y^{(i)}, s_y^{(i)})$; backprop into θ, ϕ only; then $\bar{\theta} \leftarrow m\bar{\theta} + (1 - m)\theta$.

Masking (per image)

targets M	4
target area scale	(0.15, 0.20)
target aspect ratio	(0.75, 1.50)
context area scale	(0.85, 1.00)
context aspect ratio	1
overlap	removed from context
resulting $ \text{ctx} /N$	≈ 0.25 at $G=14$

Diagnostic thresholds

std / $(1/\sqrt{D})$ healthy	0.7–1.0
partial collapse	0.05–0.30
collapsed	< 0.05
eff. rank healthy	> 0.5D
alignment (collapsed)	0 (looks perfect)
uniformity (collapsed)	0 (worst possible)
loss	tells you nothing

Training

optimiser	AdamW
LR	warmup, then cosine
weight decay	0.04 $\rightarrow$ 0.4
EMA m	0.996 $\rightarrow$ 1.0 linear
target LN	stateless, no affine
loss (code)	smooth_l1
loss (paper)	squared ℓ_2
loss (V-JEPA)	ℓ_1
predictor width	384, fixed
predictor depth	6 (B) / 12 (L,H) / 16 (G)
dropout, drop-path	0
grad clipping	none (10.0 in V-JEPA)
augmentation	RandomResizedCrop only

Four things the paper omits

1. stateless LayerNorm on the targets
2. the loss is Huber, not squared ℓ_2
3. weight decay ramps on a *cosine*
4. positions are fixed 2-D sin-cos

Ablation deltas, ranked by how much they cost you. All ImageNet-1%.

change	from	to	source
$M = 4 \rightarrow M = 1$ target block	54.2	9.0	Table 10
multi-block $\rightarrow$ rasterised masking	54.2	15.5	Table 6
latent targets $\rightarrow$ pixel targets	66.9	40.7	Table 7
target scale (0.15, 0.2) $\rightarrow$ (0.075, 0.2)	54.2	19.2	Table 8
mask target-encoder output $\rightarrow$ input	67.3	56.1	Table 11
context (0.85, 1.0) $\rightarrow$ (0.40, 1.0)	54.2	31.2	Table 9
predictor depth 12 $\rightarrow$ 6	66.9	64.0	Table 12
predictor width 384 $\rightarrow$ 1024	70.7	68.4	Table 14

All rows from Assran et al., I-JEPA, ICCV 2023. Rows are not directly comparable to each other — they use different architectures and epoch budgets — but within a row the comparison is controlled.

A closing note on how to use the citations. Every footnote in this book points at a specific table, section or file, not just a paper. That is deliberate: the fastest way to become good at this is to check somebody's claim and find that they slightly overstated it. Several of the trap boxes exist because I did exactly that. Where I could not pin something down — the numeric value of ϵ in RankMe's Eq. (2), for instance, which the paper never states — I have said so rather than quietly supply a plausible number.